

Quantum Fault Tolerance Meets Toom's Contours

Michael Ben-Or David Ponnarovsky

June 8, 2026

1 Notations

We follow the standard notations commonly used in the literature. For an integer n , we denote by $[n]$ the set of integers $\{1, 2, \dots, n\}$. To present a countable set, which might or might not be finite, where the number of elements does not matter, we use curly brackets with a subscript denoting the first index of the set. For example, $\{u_i\}_{i=1}$ represents the set $u_1, u_2, \dots$; For finite sets, we might add a superscript to present the size of the set. We use a colon inside curly brackets to specify a set of elements satisfying a condition, i.e., for sets A, B , the notation $\{a : a \in A, a \in B\}$ equals $A \cap B$. Curly brackets without subscripts or conditions refer to singletons. For example, for a given integer i , the set $\{u_i\}$ is the set that contains only the element u_i . For an integer i , we denote by 1_i the vector consisting of one at the i th coordinate and zero elsewhere.

When not otherwise mentioned, we consider quantum states over qubits, each lying in a two-dimensional Hilbert space $\mathcal{H}_2$, and denote by $\mathcal{H}_{2^n}$ the Hilbert space of an n -qubit system. Pure quantum states and their dual presentations are denoted by $|\psi\rangle, \langle\psi|$, and in the matrix representation, we use $|\psi\rangle\langle\psi|$ to denote its density matrix. We will use ρ to denote a mixed state, whose density matrix is a trace one PSD matrix.

Below, a semi proof that in CSS code partial correction $Z^e \rightarrow Z^{\hat{e}}$ doesn't add bit flip errors. It might adds a global phase. I should enter it in a section about CSS coding.

$$\begin{aligned} & X^f Z^e |w + C_z^\perp\rangle \\ &= X^f Z^e X^w |C_z^\perp\rangle \\ \text{apply } H^{\otimes n} &\mapsto Z^f X^e Z^w |C_z\rangle \\ &= (-1)^{e \cdot w} Z^f Z^w X^e |C_z\rangle \\ \text{partial correction } X^{e+\hat{e}} &\mapsto (-1)^{(e+\hat{e})(f+w)} (-1)^{e \cdot w} Z^f Z^w X^{\hat{e}} |C_z\rangle \\ \text{apply } H^{\otimes n} &\mapsto (-1)^{(e+\hat{e})(f+w)} (-1)^{e \cdot w} X^f X^w Z^{\hat{e}} |C_z^\perp\rangle \\ &= (-1)^{(e+\hat{e})(f+w)} (-1)^{e \cdot w} (-1)^{\hat{e} \cdot w} X^f Z^{\hat{e}} X^w |C_z^\perp\rangle \\ &= (-1)^{(e+\hat{e})f} X^f Z^{\hat{e}} |w + C_z^\perp\rangle \end{aligned}$$

2 Computation Model

Informally, a quantum circuit is a placement of a collection of quantum gates, taken from a finite set, at different space-time locations. Each such location records when the gate is applied and on which qubits it acts. Formally:

Definition 2.1 (Quantum Circuit). A *space-time location* is a pair $l = (t, S)$, where t is a time coordinate and $S = (s_1, \dots, s_\Delta)$ is an ordered tuple of qubit indices. For a finite collection of quantum gates

$$\mathcal{G} \subset \left\{ g: L(\mathcal{H}_2^{\otimes \Delta}) \rightarrow L(\mathcal{H}_2^{\otimes \Delta}) \right\},$$

and integers $w, d \in \mathbb{N}$, a quantum circuit C of depth d and width w is a collection of tuples

$$C = \{u_i = (g_i, l_i)\}_i,$$

where each $g_i \in \mathcal{G}$ and each location $l_i = (l_{i,1}, l_{i,2})$ belongs to $\mathbb{Z}_d \times \mathbb{Z}_w^\Delta$. We call the tuples u_i the *gates of C* . They must satisfy the following compatibility condition: if two gates have the same time coordinate, then their supports are disjoint. Namely, for $u_i = (g_i, l_i), u_j = (g_j, l_j) \in C$, the equality $l_{i,1} = l_{j,1}$ implies that the sets of qubit indices appearing in $l_{i,2}$ and $l_{j,2}$ are disjoint. We denote by

$$\text{Loc}(C) := \{l_i : u_i = (g_i, l_i) \in C\}$$

the set of locations occupied by the gates of C . We define the t -*layer* of C to be its restriction to gates at time t :

$$C|_t := \{u_i = (g_i, l_i) : u_i \in C \text{ and } l_{i,1} = t\}.$$

To define the action of a quantum circuit C on a given mixed state, we first associate to C a directed acyclic graph.

Definition 2.2 (Associated Computation DAG). Let $C = \{u_i\}_i$ be a quantum circuit with depth d and width w . We define the associated computation DAG of C , denoted by π_C , to be the directed acyclic graph $\pi_C = (\mathcal{U}, E)$ as follows:

1. The vertex set is the gate set of C , namely $\mathcal{U} = \{u_i\}_i$.
2. For $u_i = (g_i, l_i)$ and $u_j = (g_j, l_j)$, there is a directed edge $(u_i, u_j) \in E$ if and only if j is one of the minimizers of

$$\{l_{k,1} : (g_k, l_k) \in \mathcal{U}, l_{k,1} > l_{i,1}, l_{k,2} \cap l_{i,2} \neq \emptyset\}.$$

Equivalently, u_j is one of the earliest later gates whose support intersects the support of u_i . There may be more than one such minimizer, so the out-degree may exceed one.

Suppose that $J = (j_1, j_2, \dots, j_k)$ is an ordering of the gate indices that agrees with a topological order induced by π_C . Then we define the associated partial circuits $C_J = \{C_{J,i}\}_{i=1}^k$ by

1. $C_{J,1} = \{u_{j_1}\}$.
2. For $1 \leq i < k$, set

$$C_{J,i+1} = C_{J,i} \cup \{u_{j_{i+1}}\}.$$

We call $C_{J,i}$ the partial computation circuit up to step i .

Definition 2.3. Let ρ be a density matrix over w qubits, and let $C = \{u_i\}_{i=1}$ be a quantum circuit with depth d and width w . For any density matrix σ , quantum gate $g \in \mathcal{G}$, and location l , the application of the gate-location tuple (g, l) on σ , denoted by $(g, l) \circ \sigma$, is the application of g on σ where wires are ordered such that the first qubit in the input to g is $l_{2,1}$, the second is $l_{2,2}$, and so on. The result of the application of C on ρ , denoted by $C \circ \rho$, is given by iterative applications of the gates $\{u_i\}_{i=1}$ on ρ , according to a topological order induced by π_C .

We say that $\rho_1, \rho_2, \dots, \rho_k$ is a computation prefix if $\rho_1 = \rho$ and $\rho_{i+1} = u'_i \circ \rho_i$, where $\{u'_i\}_{i=1}$ is a prefix of a topological ordering of $\{u_i\}_{i=1}$. If $\{u'_i\}_{i=1}$ contains all the gates in $\{u_i\}_{i=1}$, then we call $\rho_1, \rho_2, \dots, \rho_k$ a running.

Definition 2.4 (C subjected to $\mathcal{E}$ every m computational layers). Let $C = \{u_i\}_i$ be a quantum circuit with depth d and width w , let $m \in \mathbb{N}$, and set

$$d_m := \left\lceil \frac{d}{m} \right\rceil.$$

Let $\mathcal{L} \subset [d_m] \times [w]$ be a set of locations, and let $\mathcal{E}$ be a quantum circuit, such $\text{Loc}(\mathcal{E}) = \mathcal{L}$. We name $\mathcal{E}$ *subsets of faults*, and we name the layers of $\mathcal{E}$ *fault layers*. The circuit C *subjected to $\mathcal{E}$ every m computational layers*, denoted by $C[\mathcal{E}]_m$, is obtained by keeping the order of the computational layers of C and inserting one fault layer after every block of m consecutive computational layers. Thus the first fault layer is inserted after the computational layers $1, \dots, m$, the second is inserted after the computational layers $m+1, \dots, 2m$, and so on. Concretely, each computational gate $u_i = (g_i, (t_i, S_i))$ of C is moved to the time coordinate

$$t_i + \left\lfloor \frac{t_i - 1}{m} \right\rfloor,$$

while each fault gate $(g, (t, S)) \in \mathcal{E}$ is placed at time coordinate

$$t(m+1).$$

Equivalently, for each time index $t \in [d_m]$, the computational layers with original times

$$(t-1)m+1, \dots, \min\{tm, d\}$$

appear consecutively and are followed by the fault layer at time $b(m + 1)$. When $m = 1$, the computational gates occupy the odd time coordinates and the fault layers occupy the even time coordinates. When the parameter is omitted, we mean $m = 1$. We denote the associated computation DAG of $C[\mathcal{E}]_m$ by $\pi[C, \mathcal{E}]$.

Remark 2.5. Formally, a fault circuit is just a particular quantum circuit whose gates are interpreted as faults rather than as computational operations. Thus the distinction between a quantum circuit and a fault circuit is semantic rather than structural: both are circuits in the sense of Definition 2.1, but they play different roles in the analysis.

Example 2.6. When $m = 3$, the computational layers are sent to the times

$$1, 2, 3, 5, 6, 7, 9, 10, 11, \dots,$$

while the fault layers are sent to the times

$$4, 8, 12, \dots$$

The chapter uses two different notions of propagation. They are related, but they serve different analytical purposes. The first notion does not rewrite the circuit and does not change the time coordinates. Instead, it propagates part of the error inside a fixed running of the computation and asks what the accumulated effect looks like at a chosen step τ . In particular, this notion applies not only to a single fault, but also to a collection of faults propagated one after another according to the computation DAG. This is the right framework when one wants to speak about the size of the syndrome or of the deviation *at an actual intermediate time* of the computation.

This distinction is essential for statements such as the intermediate value theorem proved below. For example, a codeword may absorb noise during one time unit and be mapped to another codeword. If one only commutes whole fault layers forward, then one reorganizes the circuit globally and may lose the ability to point to the time at which a large syndrome first appears. The first notion of propagation avoids this problem: by propagating only part of the error while keeping the underlying time axis fixed, it lets us identify intermediate steps where a medium-size or large deviation must already be visible.

The second notion is more global. There we swap entire layers and then update the time coordinates accordingly. Its role is not to locate when a syndrome becomes large, but rather to compare different time-unfoldings of the same noisy circuit and to coarse-grain the noise into sparser effective layers. In particular, once the theorem proved using this second notion is established, any constant-depth collection of gates may be treated as a single time step for the purpose of the noise analysis: up to the corresponding effective rescaling of the noise rate, we may ignore the faults that occur during the internal application of those gates and replace them by an effective fault layer acting only before or after that constant-depth subcircuit. Moreover, this effective rescaling is correct with respect to local stochastic noise channels: a local stochastic noise channel is mapped to another local stochastic noise channel, with a rescaled noise rate.

For readability, we therefore separate the discussion into two subsections. The first subsection develops propagation inside a fixed running and is tailored to the deviation-cluster bounds, while the second develops layer propagation and is tailored to the coarse-graining argument.

2.1 Local Propagation Along a Running

Definition 2.7 (Fault propagation to a step τ). Let $C = \{u_i\}_i$ be a quantum circuit, let $J = (j_1, \dots, j_k)$ be an order agreeing with π_C , and let $u_i \in C$. Suppose that $u_i = u_{j_l}$ in this order. If $l > \tau$, we define the propagation of u_i to step τ to be the identity operator.

Otherwise, define operators $X_0, X_1, \dots, X_{\tau-l}$ recursively by setting $X_0 = u_{j_l}$ and requiring that for every $1 \leq s \leq \tau - l$,

$$X_{s-1}u_{j_{l+s}} = u_{j_{l+s}}X_s.$$

We then define the *propagation of u_i to step τ* to be the operator

$$X := X_{\tau-l}.$$

Now let $C[\mathcal{E}]$ be a noisy circuit, let J be an order agreeing with $\pi_{C[\mathcal{E}]}$, and let τ be a step in that order. For each fault gate $e \in \mathcal{E}$ that occurs no later than step τ , denote by X_e^τ its propagation to step τ . We define the propagated error at step τ by multiplying these propagated faults in reverse topological order:

$$X^\tau := \prod_{e \preceq \tau} X_e^\tau.$$

Definition 2.8 (Deviation). For a quantum circuit C , a set of faults $\mathcal{E}$, and step τ , let X^τ be the propagated error at step τ . We define the deviation of $C[\mathcal{E}]$ from C at step τ by the set of indices on which X^τ acts non-trivially.

Definition 2.9. Using the same notations as in Definition 2.8, for a set of unitary operators $\mathcal{S} = \{s_i: \mathcal{H}_2^w \rightarrow \mathcal{H}_2^w\}_{i=1}$, we define the propagated error up to stabilizers in $\mathcal{S}$ as $X^\tau[\mathcal{S}] := \min_{s \in \mathcal{S}} \{w(sX)\}$. Then, we define the deviation of $C[\mathcal{E}]$ from C at step τ up to stabilizers in $\mathcal{S}$ to be the subset of coordinates on which $X^\tau[\mathcal{S}]$ acts non-trivially.

The following definitions extend the notion of a Tanner graph to the setting of quantum circuits.

Definition 2.10 (Tanner graph of a register). Let C be a quantum circuit and let R be a register, namely a subset of qubits of C . The *Tanner graph* of R at step τ , denoted by $\mathcal{T}_{R,\tau}$, is defined as follows:

1. If at step τ the register R is encoded by a code $\mathcal{C}$, then $\mathcal{T}_{R,\tau}$ is the Tanner graph of $\mathcal{C}$.

2. Otherwise, $\mathcal{T}_{R,\tau}$ is the empty graph.

When the step is clear from the context, we omit the subscript τ .

Definition 2.11 (Generalized Tanner graph of a circuit). Let C be a quantum circuit. For registers A and B in C , define the generalized Tanner graph of their union by

$$\mathcal{T}_{A \cup B, \tau} := \mathcal{T}_{A, \tau} \cup \mathcal{T}_{B, \tau}.$$

More generally, for a partition of the qubits into registers $\{R_i\}_i$, the generalized Tanner graph of C at step τ is

$$\mathcal{T}_{C, \{R_i\}, \tau} := \bigcup_i \mathcal{T}_{R_i, \tau}.$$

When the partition and the step are clear, we abbreviate this notation to $\mathcal{T}_C$.

Definition 2.12 (Clustered deviation). Let C be a quantum circuit, let R be a register of C , and let $\mathcal{E}$ be a fault set. Let $\mathcal{T}_{R, \tau}$ be the Tanner graph of R at step τ , and let $\mathcal{D}_\tau$ be the deviation of $C[\mathcal{E}]$ from C on R at step τ . Denote by $\mathcal{T}_{R, \tau}|_{\mathcal{D}_\tau}$ the subgraph induced on the left vertices corresponding to the qubits in $\mathcal{D}_\tau$.

We say that two left vertices are connected if they share a common right neighbor. The *deviation clusters* at step τ are the connected components of $\mathcal{T}_{R, \tau}|_{\mathcal{D}_\tau}$. The size of a deviation cluster is the number of right vertices it contains.

Claim 2.13. Let C be a quantum circuit whose gates act on at most Δ qubits. Let R be a quantum register encoded by a quantum error-correcting code whose Tanner graph has left degree at most Δ_1 and right degree at most Δ_2 . Fix a fault set $\mathcal{E}$ and an order J agreeing with $\pi_{C[\mathcal{E}]}$. For a step $\tau \in J$, let $Y_1^{(\tau)}, Y_2^{(\tau)}, \dots$ denote the deviation clusters at step τ .

Suppose that at step τ_1 there is a deviation cluster $Y_i^{(\tau_1)}$. Then there exist deviation clusters Z and Z' at step $\tau_1 - 1$ such that

$$Z \cap Y_i^{(\tau_1)} \neq \emptyset, \quad |Z| \geq \frac{1}{\Delta \Delta_1 \Delta_2} |Y_i^{(\tau_1)}|,$$

and

$$Z' \cap Y_i^{(\tau_1)} \neq \emptyset, \quad |Z'| \leq \Delta \Delta_1 \Delta_2 |Y_i^{(\tau_1)}|.$$

Proof. Let u be the gate applied at step τ_1 . Passing from step $\tau_1 - 1$ to step τ_1 can modify the deviation only on qubits touched by u . Since u acts on at most Δ qubits, and each such qubit is adjacent in the Tanner graph to at most Δ_1 checks, each of which meets at most Δ_2 qubits, the gate u can interact with vertices belonging to at most

$$\Delta \Delta_1 \Delta_2$$

deviation clusters from step $\tau_1 - 1$.

Let $Y_1, \dots, Y_m$, with $m \leq \Delta\Delta_1\Delta_2$, be those deviation clusters at step $\tau_1 - 1$ that meet the neighborhood affected by u and whose union contains $Y_i^{(\tau_1)}$. Then

$$|Y_i^{(\tau_1)}| \leq \sum_{j=1}^m |Y_j| \leq m \max_{1 \leq j \leq m} |Y_j| \leq \Delta\Delta_1\Delta_2 \max_{1 \leq j \leq m} |Y_j|.$$

Hence one of these clusters, call it Z , satisfies

$$|Z| \geq \frac{1}{\Delta\Delta_1\Delta_2} |Y_i^{(\tau_1)}|.$$

By construction, $Z \cap Y_i^{(\tau_1)} \neq \emptyset$.

For the upper bound, apply the same argument to the inverse gate $u^\dagger$, which also acts on at most Δ qubits. Viewing the transition from step τ_1 back to step $\tau_1 - 1$ as the application of $u^\dagger$, we conclude that some deviation cluster Z' at step $\tau_1 - 1$ intersecting $Y_i^{(\tau_1)}$ satisfies

$$|Z'| \leq \Delta\Delta_1\Delta_2 |Y_i^{(\tau_1)}|.$$

This proves both bounds. $\square$

Corollary 2.14 (Intermediate Value Theorem). Let $\alpha = 1/(\Delta\Delta_1\Delta_2)$, and let x be an integer. Assume that at the initial step τ_o , there is no deviation cluster of size more than αx , and that at a step $\tau_1 \in J$, such that $\tau_1 > \tau_o$, there is a deviation cluster $Y^{(\tau_1)}$ of size $|Y^{(\tau_1)}| > x$. Then there is a step $\tau_\star \in J$ such that $\tau_o < \tau_\star < \tau_1$ for which there is a deviation cluster Z at step $\tau_\star$, satisfying that the size of Z is in the range $(\alpha x, x)$.

Proof. Assume for contradiction that for every step τ with $\tau_o < \tau < \tau_1$, all deviation clusters have size at most αx . In particular, this holds at step $\tau_1 - 1$. On the other hand, since $|Y^{(\tau_1)}| > x$, Claim 2.13 implies the existence of a deviation cluster at step $\tau_1 - 1$ of size at least

$$\frac{1}{\Delta\Delta_1\Delta_2} |Y^{(\tau_1)}| = \alpha |Y^{(\tau_1)}| > \alpha x,$$

which is a contradiction. Therefore there must exist an intermediate step $\tau_\star$ with $\tau_o < \tau_\star < \tau_1$ at which some deviation cluster has size in the interval $(\alpha x, x)$. $\square$

Claim 2.15. Let $\alpha = (\Delta\Delta_1\Delta_2)^{-1}$. Let C be a quantum circuit with order J and depth d , and let $\mathcal{E} \sim \mathcal{N}_{\mathcal{E}}$, so that $C[\mathcal{E}]$ is a random circuit whose gates act on at most Δ qubits. Fix a time $t \leq 2d$ and an integer $x \in \mathbb{N}$. Let A be the event that by time t there exists a deviation cluster of size greater than x . For each step $\tau \in J$, let B_τ be the event that step τ is the first step at which there exists a deviation cluster of size in the interval $(\alpha x, x)$. Then

$$\Pr[A] \leq 2wd \max_{\tau \in [2wd]} \Pr[B_\tau].$$

Proof. By Corollary 2.14, whenever the event A occurs there exists a step $\tau \in J$ with $\tau \leq 2wt$ at which there is a deviation cluster of size in the interval $(\alpha x, x)$. Hence

$$A \subset \bigcup_{\tau \in [2wt]} B_\tau \subset \bigcup_{\tau \in [2wd]} B_\tau.$$

The union bound now gives

$$\Pr[A] \leq \Pr\left[\bigcup_{\tau \in [2wd]} B_\tau\right] \leq \sum_{\tau \in [2wd]} \Pr[B_\tau] \leq 2wd \max_{\tau \in [2wd]} \Pr[B_\tau].$$

□

2.2 Layer Propagation and Time Coarse-Graining

We now switch from propagation along a fixed running to propagation at the level of entire time slices. This second notion is intentionally stronger and less local. In the previous subsection, the purpose was to say: given a fault that already occurred, what is its effect at a later step of the same computation? Here the purpose is different: we want to reorganize the circuit itself, by commuting fault layers forward and grouping them into coarser effective layers. This is the mechanism behind the later renormalization claim for local stochastic noise.

In particular, local propagation keeps the ambient circuit fixed and changes the *description of the error*. Layer propagation, by contrast, changes the *presentation of the circuit* while preserving the implemented channel. This is why the two notions should not be merged into one definition: one is adapted to tracking deviations, while the other is adapted to comparing circuits related by time-unfolding.

Definition 2.16 (Layer propagation). Let $C = \{u_i\}_{i=1}$ be a quantum circuit. For each time coordinate t , write $U_t := C|_t$. The family of non-empty t -cuts $\{U_t\}_t$ is called the *layer decomposition* of C .

Let $t < t'$ be such that U_t and $U_{t'}$ are non-empty and $U_s = \emptyset$ for every $t < s < t'$. The *forward propagation* of the layer U_t across the next layer $U_{t'}$ is the operation that replaces the pair $(U_t, U_{t'})$ by a pair $(U_{t'}, V_{t \rightarrow t'})$ satisfying

$$\prod_{u \in U_t} u \prod_{v \in U_{t'}} v = \prod_{v \in U_{t'}} v \prod_{w \in V_{t \rightarrow t'}} w.$$

The layer $V_{t \rightarrow t'}$ is called the *propagation expression* of U_t across $U_{t'}$.

More generally, if $t < t'$, the *forward propagation of U_t to time t'* is obtained by iterating the above operation through the successive non-empty cuts between times t and t' . The resulting propagated layer is denoted by $V_{t \rightarrow t'}$ and is again called the propagation expression of U_t to time t' . The resulting circuit has layer decomposition

$$\dots, U_{t-1}, U_t, U_{t+1}, \dots, U_{t'}, U_{t'+1}, \dots \mapsto \dots, U_{t-1}, U_{t+1}, \dots, U_{t'}, V_{t \rightarrow t'}, U_{t'+1}, \dots$$

After propagation we update the time coordinates so that they agree with the new layer in which each gate is applied.

Claim 2.17. Assume that every gate in C acts on at most Δ qubits. Then every circuit obtained from C by forward layer propagation also consists of gates acting on at most Δ qubits.

Proof. Propagating a gate $u \in U_i$ across a layer U_{i+1} conjugates u by the product of the gates in U_{i+1} that intersect its support. Since the gates inside U_{i+1} are pairwise disjoint, each qubit in the support of u participates in at most one gate from U_{i+1} . Therefore the support of the propagated image of u is contained in the union of the supports of at most Δ gates, each of arity at most Δ , but only one such gate can be associated with each qubit of u . Consequently the propagated image still acts on at most Δ wires. Applying this argument repeatedly proves the claim. $\square$

Definition 2.18 ($C \sim_P C'$). For quantum circuits C and C' , we write $C \sim_P C'$ if one can be obtained from the other by a finite sequence of layer propagations.

By construction, $\sim_P$ preserves the implemented channel. Indeed, if $C \sim_P C'$ and ρ is any mixed state on the input register, then

$$C \circ \rho = C' \circ \rho.$$

Definition 2.19 (Noise channel). A *noise channel* $\mathcal{N}_{\mathcal{E}}$ is a distribution over sets of fault locations.

Definition 2.20 (Local stochastic noise channel). Let $p \in (0, 1)$. We say that the noise channel $\mathcal{N}_{\mathcal{E}}$ is *local stochastic with rate p* if for every finite set of locations $S \subset \mathbb{N} \times \mathbb{N}$,

$$\Pr_{\mathcal{E} \sim \mathcal{N}_{\mathcal{E}}} [S \subset \text{Loc}(\mathcal{E})] \leq p^{|S|}.$$

Given a circuit C and a random fault set $\mathcal{E} \sim \mathcal{N}_{\mathcal{E}}$, the subjected circuit $C[\mathcal{E}]$ is itself a random variable.

Definition 2.21 (Witness family in the backward light cone). Let W be a propagation expression obtained from a block of c layers of a noisy circuit after propagating all even fault layers to the end of the block. For a subset $S \subset \text{Loc}(W)$, we define the *witness family* of S , denoted by $\mathcal{W}(S)$, to be the collection of all minimal sets

$$S' \subset \Lambda(S)$$

such that there exists a fault pattern containing S' whose propagation produces all the faults at the locations in S . Here minimality is with respect to inclusion.

Claim 2.22. Let $c \geq 2$ be even, let C be a depth- $c/2$ circuit whose gates have arity at most Δ , and let $C[\mathcal{E}]$ be the circuit obtained by inserting the faults of $\mathcal{E}$ between the computational layers. Write the layer decomposition of $C[\mathcal{E}]$ as

$$U_0, U_1, \dots, U_c,$$

where the even layers are fault layers and the odd layers are computational layers. Propagate the fault layers $U_2, U_4, \dots, U_{c-2}$ forward beyond U_{c-1} , in reverse order, and denote by $V^{(1)}$ the resulting propagation expression obtained by grouping together those propagated layers with the terminal fault layer U_c . Then for every $S \subset \text{Loc}(V^{(1)})$ the witness family $\mathcal{W}(S)$ is non-empty, and every element $S' \in \mathcal{W}(S)$ satisfies

$$S' \subset \Lambda(S) \cap \bigcup_{k=0}^{c/2} U_{2k}$$

and

$$|S'| \geq \frac{|S|}{\Delta^c}.$$

Consequently,

$$\{S \subset \text{Loc}(V^{(1)})\} \subset \bigcup_{S' \in \mathcal{W}(S)} \{S' \subset \text{Loc}(\mathcal{E})\}.$$

Proof. Suppose that the event $\{S \subset \text{Loc}(V^{(1)})\}$ occurs. Then there exists at least one set of original fault locations whose propagation produces all the faults in S . Among all such sets choose one that is minimal with respect to inclusion, and denote it by S' . By construction, $S' \in \mathcal{W}(S)$, so the witness family is non-empty.

Since every element of S is obtained by propagating some original fault through at most c layers, and propagation never leaves the backward light cone of the produced qubits, every location in S' lies in $\Lambda(S) \cap \bigcup_{k=0}^{c/2} U_{2k}$.

It remains to bound the size of S' . A single fault location can influence at most Δ^c qubits in the final layer of the block, because at each of the at most c propagation steps the support can branch to at most Δ qubits. Therefore any set of original faults capable of generating all elements of S must contain at least $|S|/\Delta^c$ fault locations. In particular, this holds for the minimal producing set S' , so $|S'| \geq |S|/\Delta^c$.

Finally, whenever $S \subset \text{Loc}(V^{(1)})$ occurs, at least one minimal producing set $S' \in \mathcal{W}(S)$ is fully contained in the realized fault set $\text{Loc}(\mathcal{E})$. This proves the event inclusion. $\square$

Claim 2.23. Let $\mathcal{N}_{\mathcal{E}}$ be a local stochastic noise channel with rate p , and let C be a quantum circuit whose gates have arity at most Δ . Fix an even integer c . Then there exists a local stochastic noise channel $\mathcal{N}_{\mathcal{E}_q}$ with rate

$$q := \left(2^{H(\Delta^{-c})\Delta^c} p\right)^{\Delta^{-c}},$$

such that

$$C[\mathcal{E}]_1 \sim_P C[\mathcal{E}_q]_{c/2}.$$

In particular, up to a constant depending only on Δ and c , the rate q behaves like $p^{\Delta^{-c}}$.

Proof. Partition the subjected circuit $C[\mathcal{E}]_1$ into consecutive blocks of c layers. Inside each block, propagate all internal even fault layers to the end of the block, exactly as in Claim 2.22. Doing this block by block yields a new circuit C' in which each block of $c/2$ computational layers is followed by a single coarse-grained fault layer. Therefore $C' = C[\mathcal{E}']_{c/2}$ for a suitable random fault set $\mathcal{E}'$, and by construction

$$C[\mathcal{E}]_1 \sim_P C[\mathcal{E}']_{c/2}.$$

It remains to prove that $\mathcal{E}'$ is local stochastic. Fix one coarse-grained propagation expression $V^{(i)}$ and a subset $S \subset \text{Loc}(V^{(i)})$. By Claim 2.22, the event $\{S \subset \text{Loc}(V^{(i)})\}$ implies that at least one witness set $S' \in \mathcal{W}(S)$ is fully contained in the original fault set $\text{Loc}(\mathcal{E})$, and every such witness satisfies $|S'| \geq |S|/\Delta^c$. Hence

$$\begin{aligned} \Pr[S \subset \text{Loc}(V^{(i)})] &\leq \Pr\left[\bigcup_{S' \in \mathcal{W}(S)} \{S' \subset \text{Loc}(\mathcal{E})\}\right] \\ &\leq \sum_{S' \in \mathcal{W}(S)} \Pr[S' \subset \text{Loc}(\mathcal{E})]. \end{aligned}$$

The channel $\mathcal{N}_{\mathcal{E}}$ is local stochastic with rate p , so each term is at most $p^{|S'|}$. Moreover, the backward light cone of each effective fault has size bounded by a constant depending only on Δ and c , so the number of candidate witnesses of size ℓ is at most

$$\binom{\Delta^c |S|}{\ell}.$$

Choosing $\ell = \lceil |S|/\Delta^c \rceil$ and using the standard entropy bound

$$\binom{\Delta^c |S|}{|S|/\Delta^c} \leq 2^{H(\Delta^{-2c})\Delta^c |S|},$$

we obtain

$$\Pr[S \subset \text{Loc}(V^{(i)})] \leq \left(2^{H(\Delta^{-2c})\Delta^c} p\right)^{|S|/\Delta^c} = q^{|S|}.$$

Since this bound is uniform over all subsets S and all coarse-grained layers, the induced random set $\mathcal{E}'$ is local stochastic with rate q . Setting $\mathcal{E}_q = \mathcal{E}'$ completes the proof. $\square$

3 The (d, d') -Dimensional Toric Code

In this section, the parameter d denotes the geometric dimension of the toric complex. This use of the letter d is local to the present section.

Definition 3.1 (d -Dimensional Toric Complex). For integers d and n , denote the group resulting from the d -directed power of the cyclic group of order n by $\mathcal{G}_n^d = \mathbb{Z}_n^{\times d}$, and denote the standard generating set of $\mathcal{G}_n^d$ by $S = \{1_i : i \in [d]\}$. We define the d -dimensional toric complex to be the hypergraph obtained from the Cayley graph

$\text{Cay}(\mathcal{G}_n^d, S)$ by taking its vertices as the 0-faces and for any $i \in [d-1]$ we define the i -faces as follow: First, for a vertex $v \in \mathcal{G}_n^d$ and for a subset of generators $S' \subset S$ of size $|S'| = i$, the i -face rooted at v and spanned by S' , denoted by $\theta_{v,S'}$, is:

$$\theta_{v,S'} := \bigcup_{I \subset S'} \left\{ \left(\prod_{g \in I} g \right) v \right\}$$

The i -faces of the complex are the collection of all the possible rooted spanned i -faces induced by $\text{Cay}(\mathcal{G}_n^d)$. We will denote them by $\mathcal{F}_i$.

Additionally, we name n and d , the *side length* and the *dimension* of the complex.

Definition 3.2 (Positive Edges and Cubes). For $u, v \in \mathcal{G}_n^d$, and $S'_1, S'_2 \subset [d]$, with sizes $|S'_1| = |S'_2| = i$, we say that the i -faces θ_{v,S'_1} and θ_{u,S'_2} are adjacent if the intersection $\theta_{v,S'_1} \cap \theta_{u,S'_2}$ is an $i-1$ -face. In addition, if θ_1 and θ_2 are i and $i-1$ -faces such that $\theta_2 \subset \theta_1$, then we say that θ_2 is a *positive edge relative to face* θ_1 if they are rooted at the same vertex. Furthermore, we say that θ_1 is a *positive cube* of θ_2 . When the i -face is clear from the context, we will abbreviate and say that θ_2 is *positive*.

Example 3.3. For example, let $v, u \in \mathcal{G}_n^d$ and $g_1 \neq g_2 \in S$. Consider the 2-face $\theta = \{v, g_1v, g_2g_1v, g_2v\}$. Then, for $x, y \in \mathcal{G}_n^d$, we say that an edge (1-face) $\{x, y\}$ is *positive relative to the face* θ if $x = v$ and y is either g_1v or g_2v .

We denote the complex by $G = (V, E, F)$, where V , E , and F are the 0, 1, and 2 faces.

Although the present paper is not organized in the language of chain complexes, the boundary operators below are the standard ones for toric codes. We use them because they give a compact description of the local constraints and make the CSS structure transparent. We do not need the full homological formalism here, but it remains the natural background framework for the code family.

Definition 3.4 (Assignments on a toric complex). Let $\mathcal{T}$ be a d -dimensional toric complex. For each i , the set of i -faces induces a vector space $\mathbb{F}_2^{|\mathcal{F}_i|}$. An element $z \in \mathbb{F}_2^{|\mathcal{F}_i|}$ is called an *assignment on the i -faces* of $\mathcal{T}$.

For a vertex v and a subset of generators $S' \subset S$ of size $|S'| = i$, let $\theta_{\mathbf{v},S'}$ denote the standard basis vector corresponding to the i -face $\theta_{v,S'}$. Thus the family

$$\{\theta_{\mathbf{v},S'} : v \in \mathcal{F}_0, S' \subset S, |S'| = i\}$$

is the standard coordinate basis of $\mathbb{F}_2^{|\mathcal{F}_i|}$.

Let z be an assignment on the i -faces. Let $v_1, \dots, v_k \in \mathcal{F}_0$ and $S'_1, \dots, S'_k \subset S$ be such that $\theta_{v_1,S'_1}, \theta_{v_2,S'_2}, \dots, \theta_{v_k,S'_k}$ are precisely the i -faces in the support of z , namely:

$$z = \sum_{i=1}^k \theta_{\mathbf{v}_i, S'_i}$$

We refer to $\theta_{v_1, S'_1}, \theta_{v_2, S'_2}, \dots, \theta_{v_k, S'_k}$ as the collection of faces that form z , or briefly, the faces form z . For $i > 0$, we define the linear map $\partial : \mathbb{F}_2^{|\mathcal{F}_i|} \rightarrow \mathbb{F}_2^{|\mathcal{F}_{i-1}|}$ such that

$$\partial \theta_{\mathbf{v}, \mathbf{S}'} := \sum_{g \in S'} \theta_{\mathbf{v}, \mathbf{S}'/g} + \sum_{g \in S'} \theta_{\mathbf{g}\mathbf{v}, \mathbf{S}'/g}$$

For $i < d$, we define the linear map $\partial^* : \mathbb{F}_2^{|\mathcal{F}_i|} \rightarrow \mathbb{F}_2^{|\mathcal{F}_{i+1}|}$ such that

$$\partial^* \theta_{\mathbf{v}, \mathbf{S}'} := \sum_{g \in S/S'} \theta_{\mathbf{v}, \mathbf{S}' \cup g} + \sum_{g \in S'} \theta_{\mathbf{g}^{-1}\mathbf{v}, \mathbf{S}' \cup g}$$

We define the *restriction of z to a vertex v* to be the linear map $|_v : \mathbb{F}_2^{|\mathcal{F}_i|} \rightarrow \mathbb{F}_2^{|\mathcal{F}_i|}$ such that:

$$\theta_{\mathbf{u}, \mathbf{S}'}|_v := \begin{cases} \theta_{\mathbf{u}, \mathbf{S}'} & v \in \theta_{\mathbf{u}, \mathbf{S}'} \\ 0 & \text{else} \end{cases}$$

We define the *front of z to a vertex v* to be the linear map $|_{v+} : \mathbb{F}_2^{|\mathcal{F}_i|} \rightarrow \mathbb{F}_2^{|\mathcal{F}_i|}$ such that:

$$\theta_{\mathbf{u}, \mathbf{S}'}|_{v+} := \begin{cases} \theta_{\mathbf{u}, \mathbf{S}'} & u = v \\ 0 & \text{else} \end{cases}$$

We say that an assignment z is *connected* if the graph that is induced by taking the faces that support z as vertices and defining the edges to be the adjacency relation between them is connected. We define the connected components of z to be its subsets such that each of them is a connected assignment. For a j -face f , we say that $f \in z$ if z has a support on $\theta_{\mathbf{v}, \mathbf{S}'}$ such that $f \in \theta_{\mathbf{v}, \mathbf{S}'}$. Consider an assignment $z \in \mathbb{F}_2^{\mathcal{F}_i}$ and a vertex $v \in \mathcal{G}_n^d$. We say that z is a *rooted assignment* at vertex v if there are subsets $S'_1, \dots, S'_k \subset S^{\times k}$ such that each of S'_j is at size i and z is decomposed as:

$$z = \sum_j^k \theta_{\mathbf{v}, \mathbf{S}'_j}$$

Put differently, z is the result of taking a collection of i -faces that are rooted at v .

Claim 3.5 (CSS structure). For every i , the maps $\partial : \mathbb{F}_2^{|\mathcal{F}_i|} \rightarrow \mathbb{F}_2^{|\mathcal{F}_{i-1}|}$ and $\partial^* : \mathbb{F}_2^{|\mathcal{F}_i|} \rightarrow \mathbb{F}_2^{|\mathcal{F}_{i+1}|}$ satisfy the usual orthogonality relation between boundary and coboundary operators. Consequently, the X -checks defined by ∂ and the Z -checks defined by ∂^* commute, and therefore the resulting toric code is a CSS code.

Proof. It is enough to check the claim on basis vectors. For a basis vector $\theta_{\mathbf{v}, \mathbf{S}'}$, the appendix calculation shows that

$$\partial(\partial \theta_{\mathbf{v}, \mathbf{S}'}) = 0.$$

The same cancellation argument applies dually to the coboundary operator. Equivalently, every $d' - 1$ face and every $d' + 1$ face overlap on an even number of d' -faces, so the corresponding X - and Z -checks commute. This is exactly the CSS condition. $\square$

We will use only this CSS property and the local geometry of the code; standard facts about distance and rate may be found in the toric-code literature, for example in [Den+02].

Definition 3.6 ((d, d') Toric Code). Let d, d' be integers such that $1 < d' < d$. For any integer n , we construct the n th instance of the quantum code family (d, d') *Toric Code* as follows: Let $G = (\mathcal{F}_0, \dots, \mathcal{F}_d)$ be the d -dimensional toric complex obtained from $\mathcal{G}_n^d$ as defined in Definition 3.1. We associate the (q)bits with the $\mathcal{F}_{d'}$, the X -checks with the $\mathcal{F}_{d'-1}$, and the Z -checks with the $\mathcal{F}_{d'+1}$. An X -check, c_x , is satisfied if the parity of (q)bits set on the d' -face containing c_x is even. Similarly, a Z -check, c_z , is satisfied if the parity, in the Hadamard basis, of the (q)bits set on the d' -faces contained in c_z is even.

Claim 3.7. For any generator subset S' of size d' , denote by $x_{S'}$ and $z_{S'}$ the following assignments:

$$x_{S'} := \sum_{u \in \langle S' \rangle} \theta_{\mathbf{u}, S'} \quad z_{S'} := \sum_{u \in \langle S/S' \rangle} \theta_{\mathbf{u}, S'}$$

Then $x_{S'}$ and $z_{S'}$ are binary representations of anticommuting logical codewords of the (d, d') -Toric code. In particular, $x_{S'} \in \mathcal{C}_X / \mathcal{C}_X^\perp$, $z_{S'} \in \mathcal{C}_Z / \mathcal{C}_X^\perp$, and $x_{S'} z_{S'} = 1$. In addition, let $X_{S'}$ and $Z_{S'}$ be the Pauli matrices obtained by multiplying each non-trivial coordinate of them by X or Z , respectively.

Proof. By definition $x_{S'}$ and $z_{S'}$ are binary assignments on the d' -dimensional faces. We start by showing that $x_{S'} \in \mathcal{C}_X$ and $z_{S'} \in \mathcal{C}_Z$:

$$\begin{aligned} \partial^- \sum_{u \in \langle S' \rangle} \theta_{\mathbf{u}, S'} &= \sum_{u \in \langle S' \rangle} \sum_{g \in S'} \theta_{\mathbf{u}, S'/g} + \theta_{\mathbf{g}\mathbf{u}, S'/g} \\ &= \sum_{g \in S'} \sum_{u \in \langle S' \rangle} \theta_{\mathbf{u}, S'/g} + \sum_{u \in \langle S' \rangle} \theta_{\mathbf{g}\mathbf{u}, S'/g} \\ &= \sum_{g \in S'} \sum_{u \in \langle S' \rangle} \theta_{\mathbf{u}, S'/g} + \sum_{u \in \langle S' \rangle} \theta_{\mathbf{u}, S'/g} = 0 \\ \partial^+ \sum_{u \in \langle S' \rangle} \theta_{\mathbf{u}, S/S'} &= \sum_{u \in \langle S' \rangle} \sum_{g \in S'} \theta_{\mathbf{u}, S/S' \cup g} + \theta_{\mathbf{g}^{-1}\mathbf{u}, S/S' \cup g} \\ &= \sum_{g \in S'} \sum_{u \in \langle S' \rangle} \theta_{\mathbf{u}, S/S' \cup g} + \sum_{u \in \langle S' \rangle} \theta_{\mathbf{g}^{-1}\mathbf{u}, S/S' \cup g} \\ &= \sum_{g \in S'} \sum_{u \in \langle S' \rangle} \theta_{\mathbf{u}, S/S' \cup g} + \sum_{u \in \langle S' \rangle} \theta_{\mathbf{u}, S/S' \cup g} = 0 \end{aligned}$$

Thus $x_{S'} \in \mathcal{C}_X$ and $z_{S'} \in \mathcal{C}_Z$, now for showing that they are not binary representations of stabilizers, it's enough to show they admit no vanishing product.

$$x_{S'} z_{S'} = \sum_{u \in \langle S' \rangle} \theta_{\mathbf{u}, S'} \sum_{u \in \langle S/S' \rangle} \theta_{\mathbf{u}, S'} = \theta_{\mathbf{e}, S'} \theta_{\mathbf{e}, S'} = 1$$

□

Definition 3.8 (The Transpose Map). Let d, d' be integers, and let $\mathcal{G}$ be a (d, d') -Toric complex generated by the generator set S . We define the *transpose map* $\phi : \mathcal{F}^{d'} \rightarrow \mathcal{F}^{d-d'}$ by acting on the basis elements as follows:

$$\phi(\theta_{\mathbf{v}, \mathbf{S}'}) \rightarrow \theta_{\mathbf{v}^{-1}, \mathbf{S}/\mathbf{S}'}$$

Notice that for $d' = d - d'$, the transpose map is a permutation over the bits.

Definition 3.9 (Hadamard-Type Gate H_ϕ). For $d' = d - d'$, we define the logical Hadamard-type gate $H_\phi : L \rightarrow L$ as follows, for any $S' \subset S$, of size d' :

$$H_\phi(X_{S'}) \rightarrow Z_{S/S'}$$

$$H_\phi(Z_{S'}) \rightarrow X_{S/S'}$$

Claim 3.10. The Hadamard-type gate H_ϕ has a fold-transversal implementation by, first applying H^n over all the physical qubits, and then permuting them by the transpose map ϕ , namely $\hat{H}_\phi = \phi \circ H^n$.

Proof. Clearly, for any subset of generators $S' \subset S$, the map ϕ sends $x_{S'}$ to $z_{S/S'}$:

$$\phi(x_{S'}) = \phi\left(\sum_{u \in \langle S' \rangle} \theta_{\mathbf{u}, \mathbf{S}'}\right) = \sum_{u \in \langle S' \rangle} \theta_{\mathbf{u}, \mathbf{S}/\mathbf{S}'} = z_{S/S'}$$

Moreover, ϕ sends binary representations of Z -stabilizers to binary representations of X -stabilizers, and vice versa. To see this, consider a $d' + 1$ dimensional face, and consider the image of the stabilizer it defines.

$$\begin{aligned} \phi(\partial^- \theta_{\mathbf{v}, \mathbf{S}'}) &= \phi\left(\sum_{g \in S'} \theta_{\mathbf{v}, \mathbf{S}'/\mathbf{g}} + \theta_{\mathbf{g}\mathbf{v}, \mathbf{S}'/\mathbf{g}}\right) \\ &= \sum_{g \in S'} \theta_{\mathbf{v}^{-1}, (\mathbf{S}/\mathbf{S}') \cup \mathbf{g}} + \theta_{\mathbf{g}^{-1}\mathbf{v}^{-1}, (\mathbf{S}/\mathbf{S}') \cup \mathbf{g}} \\ &= \partial^+ \theta_{\mathbf{v}^{-1}, \mathbf{S}/\mathbf{S}'} \end{aligned}$$

And since $\phi \circ \phi = I$, we also have that $\phi(\partial^+ \theta_{\mathbf{v}^{-1}, \mathbf{S}/\mathbf{S}'})$. Thus, $H^{\otimes n} \circ \phi$ sends Z -stabilizers to X -stabilizers and vice versa, thereby preserving the code space. Combining this with the above, $H^{\otimes n} \circ \phi$ realizes the logical form H_ϕ in the $(d, d/2)$ -Toric code. $\square$

Corollary 3.11. For the $(d, d/2)$ -Toric code, consider the gate $CZ_{S' \rightarrow S/S'} : L^{\otimes 2} \rightarrow L^{\otimes 2}$, defined to act on two encoded registers by applying $Z_{S/S'}$ on the second register, controlled by whether the logical $X_{S'}$ is turned on in the first register. Then $CZ_{S' \rightarrow S/S'}$ has a fold-transversal realization.

Proof. Consider the following circuit,

1. Apply H^n on the second register.
2. For any bit $\theta_{\mathbf{v}, \mathbf{S}'}$, apply CX from $\theta_{\mathbf{v}, \mathbf{S}'}$ in the first register into $\phi(\theta_{\mathbf{v}, \mathbf{S}'})$ in the second register.

3. Again apply H^n on the second register.

□

Definition 3.12 (The φ Rotation). Consider a permutation from the generator set to itself $\pi: S \rightarrow S$. The permutation π induces a natural isomorphism by substituting generators of the product representation with their corresponding images according to π . We define the φ_π rotation, $\varphi_\pi: \mathcal{F}^{d'} \rightarrow \mathcal{F}^{d'}$, to act on the d' -dimensional faces as follows:

$$\varphi_\pi(\theta_{\mathbf{v}, \mathbf{S}'}) := \theta_{\pi(\mathbf{v}), \pi(\mathbf{S}')}$$

Claim 3.13. Fix a permutation $\pi: S \rightarrow S$. Then φ_π satisfies the following:

1. φ_π sends the binary representation of X -stabilizers to themselves, and similarly for the binary representations of the Z -stabilizers.
2. For any generator subset $S' \subset S$, of size d' , the map φ_π sends $x_{S'}$ and $z_{S'}$ to $x_{\pi(S')}$ and $z_{\pi(S')}$ respectively.

Proof. For the first part of the claim, consider an arbitrary subset of generators $S' \subset S$ of size $d' + 1$, and notice that for any generator $g \in S'$, we have $\varphi_\pi(S'/g) = \varphi_\pi(S')/\varphi_\pi(g)$. Thus:

$$\begin{aligned} \varphi_\pi(\partial^- \theta_{\mathbf{v}, \mathbf{S}'}) &= \varphi_\pi \left(\sum_{g \in S'} \theta_{\mathbf{v}, \mathbf{S}'/g} + \theta_{\mathbf{g}\mathbf{v}, \mathbf{S}'/g} \right) \\ &= \sum_{g \in S'} \theta_{\varphi_\pi(\mathbf{v}), \varphi_\pi(\mathbf{S}'/g)} + \theta_{\varphi_\pi(\mathbf{g}\mathbf{v}), \varphi_\pi(\mathbf{S}'/g)} \\ &= \sum_{g \in \varphi_\pi(S')} \theta_{\varphi_\pi(\mathbf{v}), \varphi_\pi(\mathbf{S}')/g} + \theta_{\mathbf{g}\varphi_\pi(\mathbf{v}), \varphi_\pi(\mathbf{S}')/g} \\ &= \partial^- \theta_{\varphi_\pi(\mathbf{v}), \varphi_\pi(\mathbf{S}')} \end{aligned}$$

Similarly, we find that for any subset of generators $S' \subset S$ of size $d' - 1$:

$$\begin{aligned} \varphi_\pi(\partial^+ \theta_{\mathbf{v}, \mathbf{S}'}) &= \varphi_\pi \left(\sum_{g \in S/S'} \theta_{\mathbf{v}, \mathbf{S}' \cup g} + \theta_{\mathbf{g}^{-1}\mathbf{v}, \mathbf{S}' \cup g} \right) \\ &= \sum_{g \in S'} \theta_{\varphi_\pi(\mathbf{v}), \varphi_\pi(\mathbf{S}' \cup g)} + \theta_{\varphi_\pi(\mathbf{g}^{-1}\mathbf{v}), \varphi_\pi(\mathbf{S}' \cup g)} \\ &= \sum_{g \in S/\varphi_\pi(S')} \theta_{\varphi_\pi(\mathbf{v}), \varphi_\pi(\mathbf{S}') \cup g} + \theta_{\mathbf{g}^{-1}\varphi_\pi(\mathbf{v}), \varphi_\pi(\mathbf{S}') \cup g} \\ &= \partial^+ \theta_{\varphi_\pi(\mathbf{v}), \varphi_\pi(\mathbf{S}')} \end{aligned}$$

So, we have that φ_π sends binary representations of stabilizers to representations of stabilizers of the same type. For the second part, observe that under the automorphism

induced by π , the subgroup $\langle S' \rangle$ maps to $\langle \pi(S') \rangle$, Therefore:

$$\begin{aligned}\varphi_\pi(x_S) &= \varphi_\pi \left(\sum_{u \in \langle S' \rangle} \theta_{u, S'} \right) = \sum_{u \in \langle \pi(S') \rangle} \theta_{u, \pi(S')} = x_{\pi(S')} \\ \varphi_\pi(z_S) &= \varphi_\pi \left(\sum_{u \in \langle S/S' \rangle} \theta_{u, S'} \right) = \sum_{u \in \langle S/\pi(S') \rangle} \theta_{u, \pi(S')} = z_{\pi(S')}\end{aligned}$$

□

Corollary 3.14. For a $d' = d - d'$ and a fixed generator set $S' \subset S$, at size d' , consider the logical $H < ++ >$

Definition 3.15 (The Nice Encoding). Consider $(d, d/2)$ -Toric code and fix a subset $S' \subset S$, at size d' . We define the *nice encoding* to be the subsystem code, which uses $x_{S'}$ to encode a single logical qubit. Namely, the logical codewords are:

$$\begin{aligned}|\bar{0}\rangle &= |\mathcal{C}_Z^\perp\rangle \\ |\bar{1}\rangle &= |x_{S'} + \mathcal{C}_Z^\perp\rangle\end{aligned}$$

Claim 3.16. The nice encoding, admit fold-transversal realization of X , CX and CZ . Moreover, given a resource state, the nice encoding has a fold-transversal realization of the logical **S** gate.

Proof. We realize the gates as follow:

1. The logical X gate is implemented by applying the physical X operator over any non-trivial coordinate of $x_{S'}$.
2. The CX gate is realized by applying physical CX pair-wise.
3. For the CZ gate, consider the permutation $\pi : S \rightarrow S$, such that it maps the generators in S' to its complement, namely $\pi(S') = S/S'$. We first swap the qubits in the first register according to φ_π , then apply $CZ_{S' \rightarrow S/S'}$, and eventually invert the permutation on the first register.

Since, by Claim 3.13, φ_π acts trivially on stabilizers and sends $X_{S'}$ to $X_{S/S'}$, we get that the second step $CZ_{S' \rightarrow S/S'}$ conditions $Z_{S'}$ application on whether the first register initially contained $X_{S'} |\bar{0}\rangle = |\bar{1}\rangle$. The final inversion doesn't impact the second register.

4. For the **S** gate, we use the gate teleportation implementation, which uses logical CX , logical CZ , and the logical resource state $|\mathbf{S}\rangle$ defined by:

$$|\mathbf{S}\rangle := \frac{1}{\sqrt{2}} (|\bar{0}\rangle + i |\bar{1}\rangle)$$

Figure 1 gives a description of the logical gate teleportation protocol.

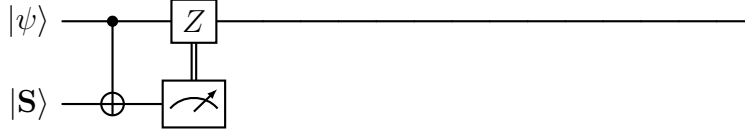


Figure 1: Simulating the **S**-gate, using the $|\mathbf{S}\rangle$ resource state.

□

Definition 3.17 (Decoding Gadget). Let C be a quantum circuit, and let u be a gate in C with fan-in at most Δ . We say that u is a *decoding gadget* if it has the following form:

1. The wires of u can be decomposed into two subsets A and B such that the wires in A are reset wires, and there exists an encoded register R in C such that the wires in B are all set in R .
2. For a set of faults $\mathcal{E}$, there is a Pauli gate u' that acts only on B such that the action of u in $C[\mathcal{E}]$ equals u' . Put differently, the quantum circuit obtained from $C[\mathcal{E}]$ by replacing u with u' equals $C[\mathcal{E}]$.

Definition 3.18 (Unitary Decoding Representation). Let $C = \{u_i\}_{i=1}$ be a quantum circuit, and let $\mathcal{U} = \{u'_i\}_{i=1} \subset C$ be a subset of the gates in C that are decoding gadgets. For a set of faults $\mathcal{E}$, we define the *unitary decoding representation* of $C[\mathcal{E}]$ to be the quantum circuit obtained by replacing each of the gates in $\mathcal{U}$ with their corresponding Pauli gate according to $\mathcal{E}$.

Definition 3.19 (Register coordinate system). Let $C = \{u_i\}_i$ be a quantum circuit of width w with registers $\mathcal{R} = R_1, \dots, R_k$, each of length at most n . For each register R_i , define its identifier by $\mathbf{1}_{R_i} = \mathbf{1}_i$. We define the map

$$\phi_{\mathcal{R}}: \mathbb{Z}_w \rightarrow \mathbb{Z}_R \times \mathbb{Z}_n$$

by setting

$$\phi_{\mathcal{R}}(x) = (\mathbf{1}_{R_j}, x'),$$

where R_j is the register containing x , and x' is the position of that qubit within R_j .

The presentation of C in the *register coordinate system* is obtained by replacing every spatial coordinate by its image under $\phi_{\mathcal{R}}$. Thus, if $u_i = (g_i, l_i) \in C$ and the space-location part of l_i is $l_{i,2} = (l_{i,2}^{(1)}, \dots, l_{i,2}^{(\Delta)})$, then we replace it by

$$(\phi_{\mathcal{R}}(l_{i,2}^{(1)}), \dots, \phi_{\mathcal{R}}(l_{i,2}^{(\Delta)})).$$

We call the first coordinate of $\phi_{\mathcal{R}}(x)$ the *register coordinate* and the second the *inner coordinate*.

4 Infinite Torics

Definition 4.1 (Shifting in Registers System). Consider the register coordinate system induced by a quantum circuit of width w , with R registers, each of length at most n . For an integer $r \in \mathbb{Z}$, we define the r -shifting function $\text{Sh}_r: (\mathbb{Z}_R \times \mathbb{Z}_n)^{\times \Delta} \rightarrow (\mathbb{Z}_R \times \mathbb{Z})^{\times \Delta}$ to act on space-location coordinates as follows:

$$\begin{aligned} \text{Sh}_r(l_1, \dots, l_k) &= (\text{Sh}_r(l_1), \dots, \text{Sh}_r(l_k)) \\ \text{Where } \text{Sh}_r(l_i) &= \begin{cases} l_i + r & \text{if } l_i \text{ is an inner coordinate} \\ l_i & \text{else} \end{cases} \end{aligned}$$

Definition 4.2. Let n be an integer, and let $C = \{u_i\}_{i=1}$ be a quantum circuit with registers $\mathcal{R} = R_1, \dots, R_k$ such that each register R_i is a toric encoding with a side length n . In addition, all the decoding gadgets of C are sweep rule gadgets; let us denote them by $\mathcal{U}$. We define the ∞ -extension of C , denoted by C^∞ , as follows:

1. For a register $R_i \in \mathcal{R}$, denote by d_i, d'_i the dimension of the toric complex that induces the code, and the dimension of the faces that are associated with the bits. Each register R_i in $\mathcal{R}$ is mapped to a register R'_i , which is a (d_i, d'_i) -dimensional, infinite side length, toric encoding. Since the mapping between registers is one-to-one, we use the identifier $\mathbf{1}_{R_i}$ to identify also R'_i .
2. Each gate $u_i \in C/\mathcal{U}$ is mapped to:

$$u_i = (g_i, l_i) \mapsto \bigcup_{j=-\infty}^{\infty} \{(g_i, (\text{Sh}_{jn}(l_i)))\}$$

3. Each gate $u \in \mathcal{U}$ is mapped to the collection of sweep rule gadgets $\{v_j\}_{j=-\infty}^{\infty}$ such that each of them has the same time coordinate as u , is in the register that corresponds to the image of u 's register, and is rooted at $\text{root}(v) + jn$.

For a set of faults $\mathcal{E} = \{f_i\}_{i=1}$, we denote by $C^\infty[\mathcal{E}^\infty]$ the ∞ -extension of $C[\mathcal{E}]$.

Claim 4.3. Let $u = (g, l)$, and let $v_0, v_{\pm 1}$ be its translates in the ∞ -extension rooted at $\text{root}(u)$ and $\text{root}(u) \pm n$. Then

$$v_0 v_{\pm 1}|_{[w]} = u.$$

Proof. The gates v_0 and $v_{\pm 1}$ are copies of the same local Pauli operator acting on translates of the same finite pattern. When restricted to the fundamental domain $[w]$, all factors outside $[w]$ disappear, and the remaining action coincides with the original gate u . Thus the product of the relevant translates restricts exactly to u . $\square$

Definition 4.4. For a quantum circuit C , of the form defined in Definition 4.2, and a set of faults $\mathcal{E}$, denote:

$$\begin{aligned} \rho &:= C[\mathcal{E}] |0\rangle\langle 0| C[\mathcal{E}]^\dagger \\ \rho_\infty &:= C^\infty[\mathcal{E}^\infty] |0\rangle\langle 0| (C^\infty[\mathcal{E}^\infty])^\dagger \end{aligned}$$

We say that a quantum circuit C is ∞ -compatible if, for any set of faults $\mathcal{E} = \{f_i\}_{i=1}$, it holds that:

$$\rho = \mathbf{Tr}_{/[w]}(\rho_\infty)$$

Claim 4.5. Let C be quantum circuit of the form defined in Definition 4.2. Then C is ∞ -compatible.

Proof. It is enough to prove the claim for circuits in their unitary decoding representation. We prove it by induction on the number of the gates.

1. Base. For an empty circuit, ρ_∞ is just the infinite string of zeros; thus, the restriction of it to $[w]$ locations, namely $\mathbf{Tr}_{/[w]}(\rho_\infty)$, yields a single matrix element corresponds to the w -length string of zeros, which is exactly ρ .
2. Step. Assume the claim holds for every circuit with fewer than τ gates, and consider a circuit C with τ gates. Denote by C' the circuit obtained from C by erasing its last gate; then C' has $\tau - 1$ gates. Denote by ρ' and ρ'_∞ the output states of $C'[\mathcal{E}]$ and $(C')^\infty[\mathcal{E}^\infty]$, respectively. By the induction assumption we have:

$$\rho' = \mathbf{Tr}_{/[w]}(\rho'_\infty)$$

Denote the last gate in C by u , and by $I = \{v_j\}_{j=-\infty}^\infty$ the collection of gates in C^∞ to which u is mapped. Let us split into cases, depending on whether u is or is not a decoding gate.

- (a) Suppose that u is not a decoding gate. Let g and l be the gate element and the location of u . By the definition of ∞ -extension, I equals:

$$I = \bigcup_{j=-\infty}^\infty \{(g, (\text{Sh}_{jn}(l)))\}$$

So there is only one gate in I , corresponding to $j = 0$, that has support on qubits with spatial location in the range $[w]$. Denote that gate by v_0 .

$$\begin{aligned} \mathbf{Tr}_{/[w]}(\rho_\infty) &= \mathbf{Tr}_{/[w]} \left(\prod_{j=-\infty}^\infty v_j \rho'_\infty \prod_{j=-\infty}^\infty v_j^\dagger \right) \\ &= v_0 \mathbf{Tr}_{/[w]} \left(\prod_{j=-\infty, j \neq 0}^\infty v_j \rho'_\infty \prod_{j=-\infty, j \neq 0}^\infty v_j^\dagger \right) v_0^\dagger \\ &= v_0 \mathbf{Tr}_{/[w]} \left(\prod_{j=-\infty, j \neq 0}^\infty v_j^\dagger \prod_{j=-\infty, j \neq 0}^\infty v_j \rho'_\infty \right) v_0^\dagger \\ &= v_0 \mathbf{Tr}_{/[w]}(\rho'_\infty) v_0^\dagger \\ &= v_0 \rho' v_0^\dagger \\ &= \rho \end{aligned}$$

- (b) In the case that u is a decoding gate, observe that there are at most two gates in I that have support on the qubits with space-location in $[w]$. If there is exactly one, then repeating the non-decoding gate case while setting this gate as v_0 gives the desired result. Else, in the case where two gates act non-trivially on qubits at $[w]$, denote those gates by v_0, v_1 . Since we assumed that C is given in its unitary decoding representation, it holds that v_0, v_1 are Paulis, in particular a product operator. Denote their product decompositions by $v_0 = \prod_k v_0^{(k)}$ and $v_1 = \prod_k v_1^{(k)}$. Thus, by repeating the non-decoding case, but extracting from the infinite product only the terms in v_0 and v_1 that act non-trivially at $[w]$, we get:

$$\text{Tr}_{/[w]}(\rho_\infty) = v_0 v_1|_{[w]} \rho' (v_0 v_1|_{[w]})^\dagger$$

Moreover, since v_0 and v_1 are translates of u in locations $\text{root}(u)$ and $\text{root}(u) \pm n$, then by Claim 4.3, we have that $u = v_0 v_1|_{[w]}$; therefore:

$$\begin{aligned} \text{Tr}_{/[w]}(\rho_\infty) &= v_0 v_1|_{[w]} \rho' (v_0 v_1|_{[w]})^\dagger \\ &= u \rho' u^\dagger \\ &= \rho \end{aligned}$$

□

The computation DAG lifts naturally to the ∞ -extension: gates that arise as translates of the same local gate act on disjoint translated supports inside different periods, so they may be executed simultaneously. The same periodic lifting applies to the underlying geometric objects, passing from the finite toric complex to its infinite cover.

5 Errors Clusters

Definition 5.1 (The standart embedding). Consider the d -dimensional infinite grid $\mathcal{G}^d$. The *standard embedding* $\mathcal{G}^d \rightarrow \mathbb{R}^d$ is given by fixing a vertex in $\mathcal{G}^d$ to be sent to the origin and then mapping the generators of the grid to the standard basis. We extend notation as follows: for a vertex $v \in \mathcal{G}^d$, a generator $g \in S$, and a coordinate $i \in [d]$, such that under the embedding $g \rightarrow i$, we denote by $v_i = v_g$ the i 'th coordinate of the image of v .

Definition 5.2 (Potential Walls). We define the *potential walls* to be the $d + 1$ functions $L_1, L_2, \dots, L_{d+1}$, from the vertices of $\mathcal{G}_n^d$ to the reals as follow:

1. For any coordinate $i \in [d]$, we define $L_i: \mathcal{G}^d \rightarrow \mathbb{R}$ as $L_i(v) := v_i$. In addition, for the generator $g \in S$ which is mapped to i , i.e $g \rightarrow i$, we identify $L_g = L_i$.
2. We define $L_{d+1}: \mathcal{G}^d \rightarrow \mathbb{R}$ by $L_{d+1}(v) := \sum_{i=1}^d v_i$.

Consider an assignment z over the faces. For a vertex $v \in z$, and a potential wall $l \in \{L_1, \dots, L_{d+1}\}$, we say that v is a *local maximum* of l in z if for any neighbor u of v , such that $u \in z$, we have $l(v) \geq l(u)$. We say that v is a *strong local maximum* if the inequality is strict, namely $l(v) > l(u)$. When the assignment z is clear from the context, we might omit it and briefly say that v is a local maximum of l .

With the assistance of potential walls, we can discuss the endpoint of a bunch of bits, which will soon become error clusters.

Definition 5.3 (Size()-Function). Consider a (d, d') -Toric complex $\mathcal{G}$, and let U be a subset of vertices, $U \subset \mathcal{G}$. We define the *size* of U to be:

$$\mathbf{Size}(U) := \sum_i^{d+1} \max_{v \in S} L_i(v)$$

Furthermore, we extend the notion to binary assignment over the i th faces of a complex $z : \mathcal{F}^{(i)} \rightarrow \mathbb{F}_2$ by associating $\mathbf{Size}(z)$ with the size of the vertex subset supporting z .

Claim 5.4. Let $\mathbf{U} = \{U_1, U_2, \dots, U_k\}$ where $U_i \subset \mathbb{R}^d$, introduce an edge between U_i and U_j iff $U_i \cap U_j \neq \emptyset$. Assume that $\mathbf{U}$ is connected. Then $\mathbf{Size}(\cup_{i=1}^k U_i) \leq \sum_{i=1}^k \mathbf{Size}(U_i)$.

Proof. It suffices to prove that if $R_1 \cap R_2 \neq \emptyset$, then $\mathbf{Size}(R_1 \cup R_2) \leq \mathbf{Size}(R_1) + \mathbf{Size}(R_2)$. Let $a_{i,j} = \max_{v \in R_j} L_i(v)$. Then for any $w \in R_1 \cap R_2$ we have:

$$\begin{aligned} \mathbf{Size}(R_1 \cup R_2) &= \sum_i^{d+1} \max a_{i,1}, a_{i,2} = \sum_i^{d+1} a_{i,1} + a_{i,2} - \min a_{i,1}, a_{i,2} \\ &\leq \sum_i^{d+1} a_{i,1} + a_{i,2} - L_i(w) = \mathbf{Size}(R_1) + \mathbf{Size}(R_2) - \mathbf{Size}(w) \\ &= \mathbf{Size}(R_1) + \mathbf{Size}(R_2) \end{aligned}$$

□

Definition 5.5. We say that a Pauli operator P is an *X-type error* if it consists of a multiplication of single-qubit Pauli X and identities. Similarly, we say that P is a *Z-type error* if it consists of a multiplication of single-qubit Pauli Z and identities. For a binary assignment over the d' -faces, $z : \mathcal{F}^{(d')} \rightarrow \mathbb{F}_2$ we say that z is the *underline assignment* of P if z is the support of the non-trivial coordinates of P . We extend the size function to X/Z -types errors as follow:

1. If P is a X -type error, then $\mathbf{Size}(P) := \mathbf{Size}(z)$.
2. Else, namely P is a Z -type error, then $\mathbf{Size}(P) := \mathbf{Size}(\phi(z))$.

We will use the notation *X/Z-type error* to specify a Pauli that is either an X -type error or a Z -type error.

Claim 5.6. The following hold:

1. $\phi \circ H^{\otimes n}$ sends X -type errors to the same size Z -type errors, and vice versa.
2. $\mathbf{CX}^{\otimes n}$ and φ send X -type errors to the same size X -type errors. Similarly, they send Z -types errors to the same size Z -type errors.

Proof. Let z be a binary assignment over the d' -faces, $z : \mathcal{F}^{(d')} \rightarrow \mathbb{F}_2$, and let P be the X -type error which z is its underline assignment. Let $Q = \phi \circ H^{\otimes n} P$, and denote by z_Q the underline assignment of Q .

$$\begin{aligned} Q|\psi\rangle &= \phi \circ H^{\otimes n} P|\psi\rangle = \phi \circ H^{\otimes n} X_z|\psi\rangle \\ &= Z_{\phi(z)} \phi \circ H^{\otimes n} |\psi\rangle = Z_{\phi(z)} |\psi_2\rangle \end{aligned}$$

Where $|\psi_2\rangle$ is a codeword. Thus, Q is a Z -type error with underline assignment $z_Q = \phi(z)$, thus:

$$\mathbf{Size}(Q) = \mathbf{Size}(\phi(z_Q)) = \mathbf{Size}(\phi^2(z)) = \mathbf{Size}(z) = \mathbf{Size}(P)$$

For $\mathbf{CX}^{\otimes n}$, we consider two registers, R_1 and R_2 . The $\mathbf{CX}^{\otimes n}$ acts transversally on the registers by applying physical $\mathbf{CX}$ gates pairwise from R_1 to R_2 , and it's clear, by commutation relations, that it duplicates X -type errors from R_1 to their counterparts on R_2 . Similarly, $\mathbf{CX}^{\otimes n}$ duplicates Z -type errors from R_2 to their counterparts on R_1 .

Lastly, for φ , denote the permutation that induces φ by $\pi : S \rightarrow S$. Observe that for any subsets of vertices $U \subset \mathcal{G}$, we have:

$$\begin{aligned} \sum_{g \in S} L_g(\varphi(U)) &= \sum_{g \in S} L_{\pi(g)}(U) = \sum_{g \in \pi(S)} L_g(U) = \sum_{g \in S} L_g(U) \\ L_{d+1}(\varphi(U)) &= \max_{v \in \varphi(U)} - \sum_{g \in S} v_g = \max_{v \in U} - \sum_{g \in S} v_{\pi(g)} = L_{d+1}(U) \end{aligned}$$

Namely, the size function is invariant to φ -rotations, $\mathbf{Size}(\varphi(U)) = \mathbf{Size}(U)$. Hence:

$$\mathbf{Size}(\varphi(P)) = \mathbf{Size}(\varphi(X_z)) = \mathbf{Size}(X_{\varphi(z)}) = \mathbf{Size}(X_z) = \mathbf{Size}(P)$$

□

Corollary 5.7. Consider two registers R_1, R_2 , and let A be a logical operator that is realized by applications of $\phi \circ H^{\otimes n}, \mathbf{CX}^{\otimes n}$, and φ . Assuming that prior to an application of A , the total error over the two registers can be decomposed into X/Z -type errors $\mathbf{P} = P_1, \dots, P_l$, then the error after the application of A over the first (second) register can be decomposed into fewer than l X/Z -type errors $\mathbf{P}' = P'_1, \dots, P'_l$ such that there is an injective $h : \mathbf{P}' \rightarrow \mathbf{P}$ such that for any $P \in \mathbf{P}'$, we have:

$$\mathbf{Size}(P) = \mathbf{Size}(h(P))$$

6 SWEEP Rule

Definition 6.1 (Sweep Rule). Let z be an assignment on the d' -faces. The *sweep rule decoding procedure* at a vertex $v \in \mathcal{G}_n^d$ is defined as follows:

Measure the checks rooted at v and look for a rooted assignment $\tilde{z}$ at v such that $(\partial\tilde{z})|_{v+} = \partial(z)|_{v+}$. In other words, the X -checks rooted at v have the same syndromes for z and $\tilde{z}$. If such an assignment exists, flip $\tilde{z}$.

Definition 6.2 (Sweep cycle). We define the *sweep cycle* to be the following procedure:

1. For each vertex, perform the sweep rule.
2. Apply the Hadamard transform $H^{\otimes n}$ followed by the ϕ -transpose, and flip the positive direction.
3. Again, apply the sweep rule at each vertex.
4. Reverse stage 2 by reordering the qubits according to the ϕ -transpose, and then apply the Hadamard transform $H^{\otimes n}$. Flip the positive direction again.

Claim 6.3. The local gates that perform the SWEEP rule are decoding gadgets.

Proof. A local SWEEP update acts on a constant-size neighborhood around a vertex, uses only local syndrome information, and may be implemented with fresh ancillas that are reset after the update. Any dependence on faults inside the gadget can therefore be pushed onto the data wires as an effective Pauli operator supported on the encoded register, while the reset ancillas carry no residual information. This is exactly the form required in the definition of a decoding gadget. $\square$

7 Shrinking

The following two claims relate the local maximum points of $L_1, \dots, L_d$ in an assignment z to the structure of the syndrome they observe. Later, we will use that relation to infer that the decoding step of the SWEEP rule can't spread the error to a vertex¹ with a higher value than the current global maximum of $L_1, \dots, L_d$. In other words, the error does not spread in the positive directions.

Claim 7.1. Let z be an assignment, and recall that the vertices in ∂z are the set of vertices adjacent to unsatisfied checks induced by z . Let v be a vertex in ∂z , and let g be a generator in S such that v is a local maximum of L_g in ∂z . Then $gv \notin \partial z$.

Proof. Suppose $gv \in \partial z$, then

$$L_g(gv) = (gv)_g = 1 + v_g = 1 + L_g(v)$$

In contradiction to v being a local maximum of L_g in ∂z . $\square$

¹To faces whose vertices have higher values of $L_1, \dots, L_d$ than the current global maximum values.

Claim 7.2. Let z be an assignment, let v be a vertex in ∂z , and let g be a generator in S such that v is a local maximum of L_g . Then, for any check associated with a $d' - 1$ face rooted at v that contains gv , it is satisfied.

Proof. Assume, through contradiction, the existence of an unsatisfied check associated with a $d' - 1$ positive face rooted at v , which contains gv . This implies $gv \in \partial z$, contradicting Claim 7.1. $\square$

For the $(d+1)$ th potential wall, we would like to have a stronger claim that implies the error cluster shrinks. For this, we start by showing that any error is equivalent, up to stabilizer addition, to an error whose left-corner edge sees a syndrome. We present two notations of local codes that describe the local view of a $(d+1)$ th edge that sees no syndrome. They are the *stabilizers code at vertex v* and the *small code at vertex v* . The first corresponds to stabilizers rooted at the vertex, while the second is the collection of local views on its positive faces that satisfy its positive checks. Then, in Claim 7.5, we consider a vertex which is a $(d+1)$ th edge and doesn't see a syndrome, meaning that its local view is in its *small code*, and show that there is a codeword of its *stabilizers code* that agrees with its positive faces. Addition by that stabilizer yields a new assignment excluding v .

Definition 7.3 (Stabilizers Code at Vertex v). We define *stabilizer code at vertex v* , denoted by Stab_v , as the restriction of the (d, d') -Toric code to the d' -dimensional faces such that their vertices are at a distance of at most one positive step in each direction from v . That is equivalent to the union of all the positive d' -dimensional faces of v with the d -dimensional faces of each of its sibling gv , excluding faces containing g as their generator. Namely:

$$\begin{aligned} \text{Stab}_v := \{ & z : \partial^- z = 0 \text{ and} \\ & z \in \text{Span}(\{ \theta_{\mathbf{v}, S'} : S' \subset S, |S'| = d' \} \\ & \bigcup_{g \in S} \{ \theta_{\mathbf{g}\mathbf{v}, S'/\mathbf{g}} : S' \subset S/g, |S'| = d' \}) \} \end{aligned}$$

The checks of the code, are of the following three types:

1. Checks that are rooted at v , each identified by choosing $d - 1$ generators from S . Namely:

$$\{ \theta_{\mathbf{v}, S'} : S' \subset S, |S'| = d - 1 \}$$

For each check, the bits that connect to it, are given by restricting the coboundary of the check to bits domain of Stab_v :

$$(\partial \theta_{\mathbf{v}, S'})|_{\text{Stab}_v} = \left(\sum_{g \in S/S'} \theta_{\mathbf{v}, S' \cup \mathbf{g}} + \sum_{g \in S'} \theta_{\mathbf{g}^{-1}\mathbf{v}, S' \cup \mathbf{g}} \right)|_{\text{Stab}_v} = \sum_{g \in S/S'} \theta_{\mathbf{v}, S' \cup \mathbf{g}}$$

2. Checks are rooted at the positive sibling of v . Consider such a sibling, let's say gv (namely the vertex that is given by stepping in the direction $g \in S$ from v).

Notice that any face rooted at gv that contains ggv cannot be contained in faces rooted at v . Thus, the checks of gv correspond to choosing subsets from S/g . Namely:

$$\bigcup_{g \in S} \{\theta_{\mathbf{g}\mathbf{v}, \mathbf{S}'} : S' \subset S/g, |S'| = d' - 1\}$$

And each check is supported by:

$$\begin{aligned} (\partial\theta_{\mathbf{g}\mathbf{v}, \mathbf{S}'})|_{\text{Stab}_v} &= \left(\sum_{g' \in S/S'} \theta_{\mathbf{g}\mathbf{v}, \mathbf{S}' \cup \mathbf{g}'} + \sum_{g' \in S'} \theta_{\mathbf{g}'^{-1}\mathbf{v}, \mathbf{S}' \cup \mathbf{g}'} \right) |_{\text{Stab}_v} \\ &= \sum_{g' \in S/S'} \theta_{\mathbf{g}\mathbf{v}, \mathbf{S}' \cup \mathbf{g}'} + \theta_{\mathbf{g}^{-1}\mathbf{v}, \mathbf{S}' \cup \mathbf{g}} \end{aligned}$$

3. Checks that their roots are at a two-step distance from v , namely vertices of the form $g, g' \in S$, where $g \neq g'$. By the arguments presented in the second type, the checks are the $d' - 1$ faces which do not have the generators on the path leading from v to them, i.e., g, g' , in their generator set.

$$\bigcup_{g, g' \in S'} \{\theta_{\mathbf{g}'\mathbf{g}\mathbf{v}, \mathbf{S}'} : S' \subset S/\{g, g'\}, |S'| = d' - 1\}$$

And for each check, its support is:

$$\begin{aligned} (\partial\theta_{\mathbf{g}'\mathbf{g}\mathbf{v}, \mathbf{S}'})|_{\text{Stab}_v} &= \left(\sum_{g \in S/S'} \theta_{\mathbf{v}, \mathbf{S}' \cup \mathbf{g}} + \sum_{g \in S'} \theta_{\mathbf{g}^{-1}\mathbf{v}, \mathbf{S}' \cup \mathbf{g}} \right) |_{\text{Stab}_v} \\ &= \sum_{g \in S/S'} \theta_{\mathbf{v}, \mathbf{S}' \cup \mathbf{g}} + \sum_{g \in S'} \theta_{\mathbf{g}^{-1}\mathbf{v}, \mathbf{S}' \cup \mathbf{g}} \end{aligned}$$

Definition 7.4 (Small Code at Vertex v). For a vertex v , we denote by $\mathcal{C}_v$ the *small code at vertex v* , which is defined to be the binary assignments over the bits (d' faces) rooted at v that satisfy the positive checks of v , namely:

$$\begin{aligned} \mathcal{C}_v &:= \{z : \partial^- z = 0 \text{ and} \\ &\quad z \in \text{Span}(\{\theta_{\mathbf{v}, \mathbf{S}'} : S' \subset S, |S'| = d'\})\} \end{aligned}$$

The checks of the code are exactly the first type checks of Stab_v .

Claim 7.5. Assume $d = 4$ and $d' = 2$. Let v be a vertex. For any $z \in \mathcal{C}_v$, there is a stabilizer $w \in \text{Stab}_v$ such that $z = w|_v$; namely, z and w agree on the faces rooted at v .

Proof. We argue that the top six rows of the generating matrix of Stab_v are exactly the generating matrix of $\mathcal{C}_v$. This means that any codeword of Stab_v is an extension of a codeword of $\mathcal{C}_v$. Denote by $H_{\mathcal{C}_v}$ and H_{Stab_v} the parity check matrices of $\mathcal{C}_v$ and

Stab_v respectively. Similarly, denote by G_{C_v} and G_{Stab_v} their generating matrices. The parity check matrices are:

$$H_{C_v} = \begin{bmatrix} 1 & 1 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 1 & 1 \\ 0 & 0 & 1 & 0 & 1 & 1 \\ 0 & 0 & 1 & 0 & 1 & 1 \end{bmatrix} \quad H_{\text{Stab}_v} = \begin{bmatrix} 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 \end{bmatrix}$$

Where we mark in blue, brown, and red the first, second, and third types of checks.

$$G_{\mathcal{C}_v} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 1 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix} \quad G_{\text{Stab}_v} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \end{bmatrix}$$

□

Conjecture 7.6. For any integers d and $d' < d$, $G_{\mathcal{C}_v}$ is contained in G_{Stab_v} . In particular, Claim 7.5 holds for any dimension.

Claim 7.7. Let z be a finite assignment. Suppose that

$$\max \{L_{d+1}(u) : u \in z\} > \max \{L_{d+1}(u) : u \in \partial^\pm z\}$$

Then there is $\tilde{z} \sim_{\mathcal{C}_X} z$ such that:

$$\max \{L_{d+1}(u) : u \in \tilde{z}\} \leq \max \{L_{d+1}(u) : u \in z\} - 1$$

Proof. Let $v_1, v_2, \dots, v_k$ be the vertices in z that achieve the maximum for L_{d+1} , namely $v_1, v_2, \dots, v_k = \arg \max_{v \in z} L_{d+1}(v)$, and assume that $v_i \notin \partial^\pm z$ for any $i \in [k]$.

Consider the i th vertex v_i . By definition, since $v_i \notin \partial^\pm z$, we have $\partial^\pm z|_{v_i} = 0$, and since v_i is a maximum point of L_{d+1} in z , it is also a local maximum, and therefore $z|_{v_i+} = z|_{v_i}$. Combining the two, we get that $z|_{v_i}$ is a codeword of the small code at v_i , namely $z|_{v_i} \in \mathcal{C}_{v_i}^\pm$. Thus, by Claim 7.5, there is a stabilizer $w_i \in \text{Stab}_{v_i}^\pm$ such that w_i and z agree on the positive faces of v_i , namely $w_i|_{v_i} = z|_{v_i}$.

Set $\tilde{z} \leftarrow z + \sum_i w_i$. Since $\sum_i w_i$ is a stabilizer, we have $\tilde{z} \sim_{\mathcal{C}_X} z$. In addition, for any $j \neq i$, it follows that $v_j \notin w_i$. Otherwise, there exists $S' \subset S$ such that $v_j \in \theta_{v_i, S'}$, and therefore $L_{d+1}(v_j) > L_{d+1}(v_i)$, in contradiction to v_i and v_j being both maximum points for L_{d+1} in $\partial^\pm z$. Thus, we have:

$$\tilde{z}|_{v_i} = (z + \sum_i w_i)|_{v_i} = z|_{v_i} + (z + w_i)|_{v_i} = 0$$

Namely, for any i , the i th vertex v_i is not in $\tilde{z}$. Hence, the vertex domain of $\tilde{z}$ is contained in the union of the vertices of z with the vertices of $w_1, w_2, \dots, w_k$, substituting $v_1, v_2, \dots, v_k$. Overall, we get:

$$\begin{aligned} \max \{L_{d+1}(u) : u \in \tilde{z}\} &\leq \max \{L_{d+1}(u) : u \in z \bigcup_i w_i / \bigcup_i v_i\} \\ &\leq \max \{L_{d+1}(u) : u \in z\} - 1 \end{aligned}$$

□

Claim 7.8. Let z be a finite assignment. Then there is $\tilde{z} \sim_{C_X} z$ such that

$$\max \{L_{d+1}(u) : u \in \tilde{z}\} = \max \{L_{d+1}(u) : u \in \partial^\pm z\}$$

Proof. Let z be a finite assignment, and let k be an integer such that $k \geq 1$ and

$$\max \{L_{d+1}(u) : u \in z\} = \max \{L_{d+1}(u) : u \in \partial^\pm z\} + k$$

For an integer $l \leq k$, consider the assignments $\tilde{z}_0, \tilde{z}_1, \dots, \tilde{z}_l$ defined as follow:

- The first element, $\tilde{z}_0 = z$.
- If the $i - 1$ th element satisfies

$$(\star) \quad \max \{L_{d+1}(u) : u \in \tilde{z}_{i-1}\} > \max \{L_{d+1}(u) : u \in \partial^\pm \tilde{z}_{i-1}\}$$

Then define the i th element, $\tilde{z}_i$, to be the result of applying the construction, given in Claim 7.7, on $\tilde{z}_{i-1}$, namely $\partial \tilde{z}_{i+1} = \partial \tilde{z}_i$ and

$$\max \{L_{d+1}(u) : u \in \tilde{z}_i\} \leq \max \{L_{d+1}(u) : u \in \tilde{z}_{i-1}\} - 1$$

If the $i - 1$ th element doesn't satisfies $\star$, then set $l = i - 1$.

- If the k th element was defined, set $l = k$.

Since $\sim_{C_X}$ is a transitive relation, we have that for any $i \in [l]$, $z \sim_{C_X} \tilde{z}_i$. If $l \neq k$, then there exists $\tilde{z}_i$ such that $\tilde{z}_i \sim_{C_X} z$ and $\tilde{z}_i$ satisfies $\star$, namely by setting $\tilde{z} = \tilde{z}_i$, we get the desired. So, it is left to handle the case where $l = k$. In that case, set $\tilde{z} = \tilde{z}_k$. By Claim 7.7:

$$\max \{L_{d+1}(u) : u \in \tilde{z}\} \leq \max \{L_{d+1}(u) : u \in \partial^\pm z\}$$

Yet, since $\tilde{z} \sim_{C_X} z$, it follows that $\partial \tilde{z} = \partial z$, So:

$$\max \{L_{d+1}(u) : u \in \partial^\pm z\} = \max \{L_{d+1}(u) : u \in \partial^\pm \tilde{z}\} \leq \max \{L_{d+1}(u) : u \in z\}$$

Therefore, we have the equality:

$$\max \{L_{d+1}(u) : u \in \tilde{z}\} = \max \{L_{d+1}(u) : u \in \partial^\pm z\}$$

□

Claim 7.9. Let z be a finite assignment and let ξ be the result of the application of the Sweep rule on it. Then:

1. For any $i \in [d]$: $\max \{L_i(v) : v \in \partial z\} = \max \{L_i(v) : v \in \partial \xi\}$
2. And for $i = d + 1$: $\max \{L_{d+1}(v) : v \in \partial z\} > \max \{L_{d+1}(v) : v \in \partial \xi\} + 1$

Proof. First, since the Sweep rule is invariant to addition by stabilizers, for any $\tilde{z}$ such that $\tilde{z} \sim_{C_X} z$, the correction applied by the Sweep rule on $\tilde{z}$ is the same as the application on z . Denote by ϕ the correction, by $\tilde{\xi}$ the result of applying the Sweep rule on $\tilde{z}$, and by w the stabilizer which is the difference between z and $\tilde{z}$, namely:

$$\begin{aligned}\xi &= z + \phi \\ \tilde{\xi} &= \tilde{z} + \phi \\ \tilde{z} &= z + w\end{aligned}$$

The $\partial^\pm$ boundary of $\tilde{\xi}$ equals:

$$\partial \tilde{\xi} = \partial (\tilde{z} + \phi) = \partial (z + \phi) = \partial \xi$$

So, in total, we have that for any $\tilde{z}$ such that $\tilde{z} \sim_{C_X} z$, it holds that $\partial \tilde{z} = \partial z$ and $\partial \tilde{\xi} = \partial \xi$. Therefore, showing that there exists $\tilde{z} \sim_{C_X} z$ for which the claim holds proves the claim for z . By Claim 7.8, there is $\tilde{z} \sim_{C_X} z$ such that:

$$\max \{L_{d+1}(u) : u \in \tilde{z}\} = \max \{L_{d+1}(u) : u \in \partial^\pm \tilde{z}\}$$

For such $\tilde{z}$, any maximum of L_{d+1} in $\partial \tilde{z}$, is also a maximum in $\tilde{z}$. Denote by $v_1, \dots, v_k$ the maximum points in $\partial \tilde{z}$. Since they are maximum points in $\tilde{z}$, it holds that for each $i \in [k]$, $\tilde{z}|_{v_i} = \tilde{z}|_{v_i+}$, and hence $\partial \tilde{z}|_{v_i} = \partial \tilde{z}|_{v_i+}$. Since $\tilde{z}|_{v_i+}$ is an assignment rooted at v_i , then v_i can suggest $\tilde{z}|_{v_i+}$, namely the condition in Sweep rule is satisfied for vertex v_i . Thus:

$$\partial \tilde{\xi}|_{v_i} = \partial \tilde{z}|_{v_i} + \partial \phi|_{v_i} = 0$$

Namely, for any i , the i th vertex v_i is not in $\tilde{\xi}$. For any vertex v denote by ϕ_v the correction made by the vertex v , namely $\phi = \sum_{v \in \partial \tilde{z}} \phi_v$. Hence, the vertex domain of $\tilde{\xi}$ is contained in the union of the vertices of $\tilde{z}$ with the vertices of $\cup_{v \in V} \phi_v$, substituting maximum points $v_1, v_2, \dots, v_k$. Overall, we get:

$$\begin{aligned}\max \{L_{d+1}(u) : u \in \partial \tilde{\xi}\} &\leq \max \{L_{d+1}(u) : u \in \partial \tilde{z} \bigcup_{v \in \partial \tilde{z}} \phi_v / \bigcup_i v_i\} \\ &\leq \max \{L_{d+1}(u) : u \in \partial \tilde{z}\} - 1\end{aligned}$$

Let v be a vertex that is a maximum of L_g in $\partial \tilde{z}$, maximum points of L_g do not see syndrome at the g direction. If and:

$$\phi_v = \sum \theta_{\mathbf{v}, \mathbf{S}'_i}$$

By Claim 7.2 $\partial\tilde{z}|_{gv} = 0$, Hence:

$$\begin{aligned}\partial\phi_v|_{gv} &= \left(\partial \sum \theta_{\mathbf{v}, \mathbf{S}'_i}\right)|_{gv} \\ &= \left(\sum_{\mathbf{S}'_i} \sum_{g' \in \mathbf{S}'_i} \theta_{\mathbf{v}, \mathbf{S}'_i/g'} + \theta_{\mathbf{g}'\mathbf{v}, \mathbf{S}'_i/g'}\right)|_{gv} \\ &= 0\end{aligned}$$

Thus if $g \in \cup \mathbf{S}'_i$, Then:

$$\theta_{\mathbf{g}\mathbf{v}, \mathbf{S}'_i/g} \in \partial\phi_v|_{gv}$$

So for $\theta_{\mathbf{g}\mathbf{v}, \mathbf{S}'_i/g} = 0$, it must appear at least twice in the sum, yet the left terms in the sum cannot contain it (they differ by the root vertex), and if there is a right term $\theta_{\mathbf{g}\mathbf{v}, \mathbf{S}'_j/g} = \theta_{\mathbf{g}\mathbf{v}, \mathbf{S}'_i/g}$, then it implies that $\mathbf{S}_j = \mathbf{S}_i$. Which is a contradiction, therefore: $g \notin \cup \mathbf{S}'_i$ and $gv \notin \phi_v$.

$$\begin{aligned}\max \left\{ L_g(u) : u \in \partial\tilde{\xi} \right\} &\leq \max \left\{ L_g(u) : u \in \partial\tilde{z} \bigcup_{v \in \partial\tilde{z}} \phi_v / \bigcup_i v_i \right\} \\ &\leq \max \left\{ L_g(u) : u \in \partial\tilde{z} \right\} + 1 - 1\end{aligned}$$

□

8 Toric Encoded Circuits and Toom's Contours

Definition 8.1 (Toric Encoded Circuit). Let C be a quantum circuit. We say that C is a *Toric encoded circuit* if the following holds:

1. There is a Toric code $\mathcal{C}$ such that the qubits of $\mathcal{C}$ can be partitioned into registers $R_1, \dots, R_k$, and, at any time, each of the registers is encoded either by the *nice encoding of $\mathcal{C}$* or by the *nice encoding of $\phi \circ H(\mathcal{C})$* ². At time $t = 0$, the registers are initialized to be either $|\bar{0}\rangle$ or encoded $|\mathbf{S}\rangle$.
2. The layers of C at even times are sweep-cycles, and the layers at odd times are realizations of one of the logical $\mathbf{H}, \mathbf{S}, \mathbf{CX}, \mathbf{X}$, or $\mathbf{Z}$ gates.

Definition 8.2 (The Boundary of Deviation $\partial\mathcal{D}_t$). Let C be a Toric encoded circuit, with registers $R_1, \dots, R_l$ and let $\mathcal{E}$ be a set of faults. Consider the subjection $C[\mathcal{E}]$, For a time t , let $A_t, B_t \subset \{R_1, \dots, R_l\}$ be the registers that encoded, at time t in $C[\mathcal{E}]$, by $\mathcal{C}$ and $\phi \circ \mathbf{H}$ respectively. Denote by $\mathcal{D}_t$ the deviation in $C[\mathcal{E}]$ at time t . We define the *X-type*, and the *Z-type boundaries* of $\mathcal{D}_t$, denoted by $\partial\mathcal{D}_t^X$ and $\partial\mathcal{D}_t^Z$, as follow:

$$\begin{aligned}\partial\mathcal{D}_t^X &:= \bigcup_{R_i \in A_t} \partial |R_{i,t}\rangle \cup \bigcup_{R_i \in B_t} \partial (\phi \circ \mathbf{H}) |R_{i,t}\rangle \\ \partial\mathcal{D}_t^Z &:= \bigcup_{R_i \in B_t} \partial |R_{i,t}\rangle \cup \bigcup_{R_i \in A_t} \partial (\phi \circ \mathbf{H}) |R_{i,t}\rangle\end{aligned}$$

²Notice that although the code space is invariant to the application of $\phi \circ \mathbf{H}$, the positive direction of the underlying Toric complex is flipped.

Definition 8.3 (Base graph G_o). Let C be a quantum circuit, at depth d , with registers $R_1, \dots, R_k$, all of which are encoded by Toric codes with the same parameters (i.e., dimension, length, etc.). Let's denote by $\mathcal{G}_1, \dots, \mathcal{G}_k$ the Toric complexes that induce the codes. We define the base graph of C , denoted by $G_o = (V_o, E_o)$ as follows:

1. The vertices V_o are tuples, where the first coordinates correspond to a vertex in the union over the vertices in $\mathcal{G}_1, \dots, \mathcal{G}_k$, and the second coordinate is associated with a time. Namely:

$$V_o := \bigcup_i V(\mathcal{G}_i) \times [d]$$

2. The edges E_o are derived from the original edges in $\mathcal{G}_1, \dots, \mathcal{G}_k$, with the addition that any two vertices in preceding times t and $t + 1$ that support qubits (faces) such that C acts non-trivially on those qubits at time t are also connected by an edge. Namely:

$$\begin{aligned} E_o := \{ \{ (v, t), (u, t') \} : & \text{ if} \\ & \text{either there exists } i \text{ such } \{v, u\} \in \mathcal{G}_i \text{ and } t' = t, t + 1 \\ & \text{or there exists } u_i = (g_i, l_i) \in C \text{ such } v, u \in l_{i,2} \text{ and } t = l_{i,1}, t' = l_{i,1} + 1 \} \end{aligned}$$

[COMMENT] Here we should add that a vertex v belongs to location l if there is a qubit f that v belongs to its associated face.

Definition 8.4 (Tooms Contour Graph of $C(\mathcal{E})$). For a quantum circuit C and set of faults $\mathcal{E}$, let us denote the base graph of $C[\mathcal{E}]$ by G_o . For each time t , denote the deviation induced by $\mathcal{E}$ at time t by $\mathcal{D}_t$. We define the *Tooms contour graph* $\mathcal{H} = (V_{\mathcal{H}}, E_{\mathcal{H}})$ as follows:

1. The vertices $V_{\mathcal{H}}$ are partitioned into two types, X -type vertices $V_{\mathcal{H},X}$ and Z -type vertices $V_{\mathcal{H},Z}$. Each of the X/Z -type corresponds to the vertices supporting the X/Z -boundary of $\mathcal{D}$. Namely:

$$\begin{aligned} V_{\mathcal{H},X} &:= \{ v_t : v_t = (v, t) \in V_o \text{ and there exists a face } f \in \partial \mathcal{D}_t^X \text{ such } v \in f \} \\ V_{\mathcal{H},Z} &:= \{ v_t : v_t = (v, t) \in V_o \text{ and there exists a face } f \in \partial \mathcal{D}_t^Z \text{ such } v \in f \} \end{aligned}$$

Similarly, the Z -type vertices $V_{\mathcal{H},Z}$ are the vertices $(v, t) \in V_o$, that satisfy that v belongs to a face associated with a check, which, at time t , belongs to the $(\partial \mathcal{D}_t)|_Z$.

We extend the action of potential walls to act on the vertices of $V_{\mathcal{H}}$ by acting on the associated points in $\mathbb{R}^d$. More concretely, for a vertex $v \in V_{\mathcal{H}}$, whose corresponding vertex on the infinite $\mathbb{R}^d$ Toric is $\tilde{v} \in \mathcal{G}$, and a potential wall L_i , the value of L_i at v is:

$$L_i(v) := L_i(\tilde{v})$$

2. The edges $E_{\mathcal{H}}$ is partitioned into two types $A_{\mathcal{H}}$ and $F_{\mathcal{H}}$, where for any logical gate g which applied in time t in $C[\mathcal{E}]$ we add edge to $A_{\mathcal{H}}$ as follow:

- (a) If g is a logical **CX** then for any physical **CX** in its realization, between faces f_1 into f_2 at time t we add:

$$\begin{aligned} & \{ \{ (u, t), (v, t+1) \} : u \in f_1, v \in f_1 \cup f_2 \text{ and } v, u \in V_{\mathcal{H}, X} \} \\ & \{ \{ (u, t), (v, t+1) \} : u \in f_2, v \in f_1 \cup f_2 \text{ and } v, u \in V_{\mathcal{H}, X} \} \end{aligned}$$

- (b) If g is logical $\phi \circ \mathbf{H}$, then for any physical **H**, in its realization, that acts on face f , we add:

$$\begin{aligned} & \{ \{ (u, t), (v, t+1) \} : u \in f, v \in \phi(f) \text{ and } v \in V_{\mathcal{H}, Z}, u \in V_{\mathcal{H}, X} \} \\ & \{ \{ (u, t), (v, t+1) \} : u \in f, v \in \phi(f) \text{ and } v \in V_{\mathcal{H}, X}, u \in V_{\mathcal{H}, Z} \} \end{aligned}$$

- (c) If g is one of logical X, Z, I or a sweep-cycle, then for any face f , in the register on which acts g , we add:

$$\begin{aligned} & \{ \{ (u, t), (v, t+1) \} : u, v \in f \text{ and } u, v \in V_{\mathcal{H}, X} \} \\ & \{ \{ (u, t), (v, t+1) \} : u, v \in f \text{ and } u, v \in V_{\mathcal{H}, Z} \} \end{aligned}$$

- (d) If g is the logical **S** on register R , using a magic state encoded in register $R_{\mathbf{S}}$, then we connect edges by following the connection rules: first applying logical **CX** from R into $R_{\mathbf{S}}$, then applying logical $\phi \circ \mathbf{H}$ on $R_{\mathbf{S}}$, and then applying logical **CX** from $R_{\mathbf{S}}$ into R .

In addition for any fault g of $\mathcal{E}$, that is applied at time t , and any faces $f_1 \notin \text{Sup}_X(g)$ and $f_2 \notin \text{Sup}_Z(g)$, we add the edges:

$$\begin{aligned} & \{ \{ (u, t), (v, t+1) \} : u, v \in f_1 \text{ and } u, v \in V_{\mathcal{H}, X} \} \\ & \{ \{ (u, t), (v, t+1) \} : u, v \in f_2 \text{ and } u, v \in V_{\mathcal{H}, Z} \} \end{aligned}$$

To construct $F_{\mathcal{H}}$, we connect any pair of vertices (v, t) , (u, t) if there exists a vertex $(w, t+1)$ such both of them are connected to it through $A_{\mathcal{H}}$. Namely:

$$F_{\mathcal{H}} = \{ e = \{u, v\} \in E_o : \text{exists } w \in V_{\mathcal{H}} \text{ such } \{v, w\}, \{u, w\} \in A_{\mathcal{H}} \}$$

We call $A_{\mathcal{H}}$ and $F_{\mathcal{H}}$ the *arrows* and *forks* edges respectively.

[COMMENT] It seems that is better not to separate between X -type and Z -type clusters in the investigation graph.

8.1 Space-Time Graph

Definition 8.5. Consider a graph $G = (V, A, F)$, where V is a set of vertices, and A and F are sets of undirected edges. For $v \in V$, denote by $\text{pre}(v) = \{u \in V : \{u, v\} \in A\}$, we extend the notion for subsets of vertices $X \subset V$ by $\text{pre}(X) = \bigcup_{v \in X} \text{pre}(v)$. Let $T: V \rightarrow \mathbb{N}$, We say that (G, T) is a time graph if and only if:

1. $\{x, y\} \in A \Rightarrow T(y) = T(x) + 1$
2. For $x, y \in V$ $\{x, y\} \in F$ if and only if there exists $z \in V$ such both $\{x, z\}, \{y, z\} \in A$.

We will call to T time-function, to A arrows, and to F forks. Observe that from the second requirement it follows that forks can connect only vertices that have the same time value (set by T).

Remark 8.6. The Toom's contour graph defined in Definition 8.4 is, by definition of its construction, a time-graph.

Definition 8.7 (History, slice.). For a time graph (G, T) and a vertex $u \in V$, let G_u be the subgraph of (V, A) induced by $\{v \in V : T(v) \leq T(u)\}$. We will denote by H_u the 'History of u ', which is the set of vertices that are connected to u in G_u . A slice is an equivalence class of the relation $u \equiv v$ if and only if $H_u = H_v$. Notice that the time function has to be constant on a slice, to see this consider vertices x, y with $T(x) > T(y)$, thus $x \notin G_y$ and therefore $x \in H_x/H_y$. So they belong to different classes. Thus, we can extend the time function and the history to slices and write for a slice K , $T(K)$ and H_K .

Claim 8.8. Consider slices K_1, K_2 such that $T(K_1) = T(K_2)$, then either $K_1 = K_2$ or H_{K_1}, H_{K_2} are disjoint.

Proof. Assume a non-empty intersection, so there exists $z \in H_{K_1} \cap H_{K_2}$ such that for any $x \in K_1$ and $y \in K_2$, there exist arrow-paths $z \rightarrow x$ and $z \rightarrow y$. Thus, for any $w \in H_{K_1}$, one can concatenate a path $w \rightarrow x \rightarrow z \rightarrow y$ and conclude that $y \in H_{K_2}$. $\square$

Definition 8.9 (The graph of slice). Let K be a slice. The graph of K , denoted by $G_K = (V_K, E_K)$, is the graph whose vertices are slices $R \subset H_K$ with $T(R) = T(K) - 1$. Two vertices are connected by an edge if and only if there is a fork whose ends belong to the slices associated with them. Namely, $S, R \in V_K$ are connected in G_K if and only if there is $\{s, r\} = f \in F$ such that $s \in S$ and $r \in R$.

Claim 8.10. G_K is connected.

Proof. Let $R, S \in V_K$ and consider $r, s \in V$ such that $r \in R, s \in S$. Recall that $R, S \subset H_K$ and therefore $r, s \in H_K$. Thus, by definition of the 'History' there exists a path, passing through arrows only, from r to s in H_K . Consider such a path that is minimal in the number of points x belong to it, with $T(x) = T(K)$. By induction on the number k of such points we prove that R and S are connected in G_K .

1. Base. $k = 0$, In this case, the path from r to s lies within $G_r (= G_s)$. Hence $H_r = H_s$, So $R = S$.
2. Induction step. There exists y on the path from r to s such that $T(y) = T(K)$. Let x be the point which precedes y on the path, and z the point that follows y . Since the values of T at the two ends of an arrow differ by 1, and since y gets the maximal T value in H_K it follows that:

- (a) $\{x, z\} \subset \text{pre}(y) \Rightarrow \{x, z\} \in F$.
- (b) The slices of x and z , say K_x and K_z has time value $T(K_x) = T(K_z) = T(K) - 1$, Namely they are both members of V_K .

Since the path connecting r, x in H_K has $k - 1$ vertices with $T(\cdot) = T(K)$, then by the induction hypothesis, R and K_x are connected in G_K , and by similar argument so are K_z and S . On the other hand, $\{x, z\} \Rightarrow \{K_x, K_z\} \in E_K$. Thus, R and S are connected in G_K .

□

Definition 8.11 (Spanned Set). Consider a subset $A \subset \mathbb{R}^d$, and $d + 1$ points $x_1, \dots, x_{d+1}$. We say that $\langle A, x_1, \dots, x_{d+1} \rangle$ is a spanned set of A if for any $i \in [d + 1]$ and $s \in A$, it holds that $L(s) \leq L_i(x_i)$ and in addition there are $v_1, \dots, v_{d+1} \in V$, not necessary different, that achieve the equalities $L_i(x_i) = L_i(v_i)$. We name $x_1, \dots, x_{d+1}$ the poles of A . Furthermore, we extend the notation to a spanned slice when A is a slice of some time-graph. Notice that saying $\langle A, x_1, \dots, x_{d+1} \rangle$ is a spanned set means that $\mathbf{Size}(A) = \mathbf{Size}(\{x_1, \dots, x_{d+1}\})$.

Observe that in the case where the subset A is a slice. Each of these poles can be associated with a vertex of G that gives the equality. If there are multiple vertices whose give equality, associate the point with one of them arbitrarily. We will use the notion of applying functions originally defined over vertices to the poles, referring to the application over the vertices associated with them. For example $\text{pre}(x_i) = \text{pre}(v)$ for the vertex $v \in A$ which satisfies $L_i(v) = L_i(x_i)$ and therefore is associated with the pole x_i . Another example is referring to x_i as a vertex, so when saying 'For $x_i \in V_k$ ' or 'For $\{x_i, u\} \in F$ ', we refer to the associated vertex or the edge connecting it to u .

Claim 8.12. Consider a time graph (G, T) , Let K be a slice in G , and let $\hat{K} = \langle K, x_1, \dots, x_{d+1} \rangle$ be a spanned slice of K such that $K \neq H_K$. Then for some integer k there exist spanned slices $\hat{K}_0 = \langle K_0, \cdot \rangle, \dots, \hat{K}_k = \langle K_k, \cdot \rangle$ and Forks $F_1, \dots, F_k$ such that:

1. For $i \in [d + 1]$ $\text{pre}_i(x_i)$ belongs to the poles $\hat{K}_0, \dots, \hat{K}_k$.
2. Introduce an edge between $\hat{K}_i$ and $\hat{K}_j$ if and only if one of the Forks $F_1, \dots, F_k$ consists of a pole of $\hat{K}_i$ and a pole of $\hat{K}_j$. Then the graph formed from $\hat{K}_0, \dots, \hat{K}_k$ is connected.
3. $\sum_{i=0}^k \mathbf{Size}(\hat{K}_i) + \sum_{i=1}^k \mathbf{Size}(F_i) \geq \sum_{i=1}^{d+1} L_i(\text{pre}_i(v_i))$

Proof. First Notice that K doesn't contain leaves in (V, A) , since a leaf is connected only to itself via arrows, and therefore its 'slice' equals to its 'History'. Thus $\text{pre}_i(x_i)$ is not empty for $i = 1, \dots, d+1$. Next consider the graph $G_K = (V_K, E_K)$ from definition 8.9. Since $\{x_i, \text{pre}_i(x_i)\}$ is an arrow $\text{pre}_i(x_i)$ belongs to a slice in V_K . for $i = 1, \dots, d+1$ denote by R_i the slice in V_K that contains $\text{pre}_i(x_i)$.

By claim 8.10 G_K is connected and therefore there exist a minimal collection $\{K_0, \dots, K_k\}$ of slices form V_K which contains $R_1, R_2, \dots, R_{d+1}$, and which is connected in G_K . We pick a set of forks $\mathbf{F} = \{F_1, \dots, F_k\}$ which realizes a spanning tree for this collection of slices. Later we will identify edges from this spanning tree with the forks from $\mathbf{F}$.

For $i = 1, \dots, d+1$ we set $\text{pre}_i(v_i)$ to be the i th pole of R_i , This assures (1). The set of remaining poles for $\hat{K}_0, \dots, \hat{K}_k$ will be equal to $\cup_{i=k}^k F_i$. This will assure (ii), We will also select poles for $F_1, \dots, F_k$ to form 'spanned forks' $\hat{F}_1, \dots, \hat{F}_k$ in such way that:

$$\sum_{i=0}^k \text{Size}(\hat{K}_i) + \sum_{i=1}^k \text{Size}(\hat{F}_i) \geq \sum_{i=1}^{d+1} L_i(\text{pre}_i(v_i))$$

This will assure (3).

Let us denote by $J_0, \dots, J_{2k}$ the enumerated union $K_0, \dots, K_k, F_1, \dots, F_k$. Observes that for any choice of $(d+1)(2k+1)$ elements $z_0^1, z_0^2, z_0^3, \dots, z_{2k}^{d+1}, \dots, z_{2k}^{d+1}$ in $\mathbb{R}^d$, such that $z_j^i \in J_j$ it holds that:

$$\sum_{i=0}^k \text{Size}(\hat{K}_i) + \sum_{i=1}^k \text{Size}(\hat{F}_i) \geq \sum_{j,i} L_i(z_j^i) \quad (1)$$

Later, we are going to take the points $\{z_j^i\}$ from the intersection of J_j 's. For convenience, let us assume that for the index subset $X \subset [2k] \cup \{0\}$, the intersection $\bigcap_{j \in X} J_j$ returns a single point. If there is more than one, then decide on the point in a way that all the chosen points simultaneously satisfy that for any index subsets Y, X , such that $X \cap Y$ and they both introduce a non-trivial intersection $\bigcap_{j \in Z} J_j : Z \in \{X, Y\}$, then:

$$\bigcap_{j \in X} J_j = \bigcap_{j \in Y} J_j$$

Now consider a non-trivial maximal intersection $\bigcap_{j \in X} J_j$, the intersection is maximal in the sense that for any $j' \notin X$ it holds that $\bigcap_{j \in X \cup \{j'\}} J_j = \emptyset$. We claim that for any i and for any such X , there is $t \in X$ such that J_t is the successor of J_j on the path to R_i for any $j \in X/\{t\}$. Assume not. Let $x, y \in X$, and denote the paths from J_x, J_y to the i th pole by $\langle J_x, J_{x_2}, \dots, R_i \rangle$, $\langle J_y, J_{y_2}, \dots, R_i \rangle$, such that $J_y \neq J_{x_2}$. If $y = x_2$, $y_2 = x$, or $x_2 = y_2 \in X$, then pick other x, y (if there is no such, we get an immediate contradiction). In that case, $\langle J_x, J_y, J_{y_2}, \dots, R_i \rangle$ is a path from J_x to R_i which is different from $\langle J_x, J_{x_2}, \dots, R_i \rangle$. Therefore, there are at least two paths from J_x to the i th pole, namely, there is a cycle in contradiction to the minimality of $K_0, \dots, K_k$.

We will choose the z_j^i by the following process. Pick a z_j^i that has not been set yet. If $J_j = R_i$, then set $z_j^i \leftarrow \text{pre}_i(x_i)$ - we call this a type-1 assignment. Otherwise, consider the path from J_j to R_i (by connectivity, it has to be such a one, and by minimality, there is exactly one). Let J_t be the successor on the path, then pick $z_j^i \in J_j \cap J_t$ - similarly, we call this assignment a type-2 assignment. Since any type-2 assignment is associated with a non-trivial intersection, and any non-trivial intersection induces a type-2 assignment for all i 's (since for all i 's, one of the J_j on its support is a successor of the rest on their path to the i th pole), we have that:

$$\begin{aligned} \sum_{j,i} L_i(z_j^i) &= \sum_{j,i,z_j^i \in \text{type-1}} L_i(z_j^i) + \sum_{j,i,z_j^i \in \text{type-2}} L_i(z_j^i) \\ &= \sum_i^{d+1} L_i(\text{pre}_i(x_i)) + \sum_{X: \bigcap_{j \in X} J_j \neq \emptyset} (|X| - 1) \sum_i^{d+1} L_i \left(\bigcap_{j \in X} J_j \right) \\ &= \sum_i^{d+1} L_i(\text{pre}_i(x_i)) \end{aligned}$$

Plugging it into eq. (1) gives the desired. $\square$

Definition 8.13. Let $\hat{K}$ be a spanned slice, and let $\hat{K}_0, \dots, \hat{K}_k$ be spanned slices in H_K , and let $F_1, \dots, F_k$ be spanned forks, such that $\hat{K}_0, \dots, \hat{K}_k, F_1, \dots, F_k$ satisfy the conditions in Claim 8.12. We call $\hat{K}_0, \dots, \hat{K}_k, F_1, \dots, F_k$ the *predecessor decomposition* of $\hat{K}$. Furthermore, we say that $\hat{K}$ is a result of a *sweep-cycle*, *faults*, or a *logical operation* if the layer at time $T(\hat{K}) - 1$ consists of sweep-cycle gates, fault gates (whose source is $\mathcal{E}$), or logical gates. The *order* of the slice is its time modulo 4, namely $\text{order}(\hat{K}) = T(\hat{K}) \bmod 4$. Observe that slices that are results of faults have an order that is either 0 or 2, whereas slices that are results of a sweep-cycle or logical operation are of orders 1 and 3, respectively.

Corollary 8.14. Consider a spanned slice $\hat{K}$, and let $\hat{K}_0, \dots, \hat{K}_k, F_1, \dots, F_k$ be the predecessor decomposition of $\hat{K}$.

1. If $\hat{K}$ is a result of ξ -sweep-cycle, then:

$$\sum_{i=0}^k \text{Size}(\hat{K}_i) + \sum_{i=1}^k \text{Size}(F_i) \geq \text{Size}(\hat{K}) + \xi$$

2. If $\hat{K}$ is a result of either faults or a logical operation, then:

$$\sum_{i=0}^k \text{Size}(\hat{K}_i) + \sum_{i=1}^k \text{Size}(F_i) \geq \text{Size}(\hat{K})$$

Proof. By Claim 5.6

1. For $g = \varphi$ and $g = \phi \circ \mathbf{H}$, the gates apply

□

Corollary 8.15. For any $i \in [k]$, denote by $\hat{K}_0^i, \dots, \hat{K}_{k_i}^i, F_1^i, \dots, F_{k_i}^i$ the be the predecessor decomposition of $\hat{K}_i$. Then if $\hat{K}$ is a result of..

[COMMENT] Same proof works when after the application of $\phi \circ \mathbf{H}$, then $\text{pre}_i(x_i) = \phi(x_i)$.

$$\begin{aligned} \sum_{i=0}^k \text{Size}(\phi(\hat{K}_i)) + \sum_{i=1}^k \text{Size}(\phi(F_i)) &\geq \sum_{i=1}^{d+1} L_i(\phi(\text{pre}_i(v_i))) \\ &= \sum_{i=1}^{d+1} L_i(\phi(\phi(v_i))) = \text{Size}(K) \end{aligned}$$

After the application of $\phi \circ \mathbf{H}$ the slices in the history of X slice are Z -slices. It seems that the most continent way to handle the case is when we map the errors to:

$$\begin{aligned} X_f &\rightarrow X_f Z_{\psi(f)} \\ Z_f &\rightarrow Z_f X_{\psi(f)} \end{aligned}$$

By that, we earn that when moving the dual system the error remains the same. For that to work it seems that we have also to say something like for $\mathcal{E}' \subset \mathcal{E}$ if things work to $\mathcal{E}$ then they also should work for $\mathcal{E}'$. But that is not true cause $\mathcal{E}$ might contains cancellations. That is a problem.

Definition 8.16. Let C be a toric encoded circuit, let $\mathcal{E}$ be a set of faults, and consider Toom's contour $G_{\mathcal{H}}$ defined by it. We extend the slice definition into the language of X/Z -type error clusters as follows: We define an X -slice to be a slice that is supported only on the vertices in $V_{\mathcal{H},X}$. Similarly, a Z -slice is a slice that is supported only on the vertices of $V_{\mathcal{H},Z}$. As in X/Z -type error clusters, we define the size function to be conditioned on the type.

$$\begin{aligned} \text{Size}(X\text{-slice } K) &= \sum_i^{d+1} \max_{v \in K} L_i(v) \\ \text{Size}(Z\text{-slice } K) &= \sum_i^{d+1} \max_{v \in K} L_i(\phi(v)) \end{aligned}$$

Claim 8.17. Let K be an X -slice, such that the logical gate preceding it is $\phi \circ \mathbf{H}$. Observes that by definition 8.4, that the slices in G_K are Z -slices.

9 Explantation of a Spanned Slice

[COMMENT] Here we follow the original proof, but change the triminalogy, m will be the number of leaves in the explanation, then in the end we conclude that the number of faults is at least m/Δ . So $|R| \leq f(\frac{m}{\Delta})$.

Definition 9.1. Consider a Tooms contour graph induced by Toric encoded circuit, with sweep-cycle parameter $\xi = 2$, and let α, β be constants and $\mathcal{X}_0, \mathcal{X}_1, \mathcal{X}_2, \mathcal{X}_3 \in \mathbb{R}$ be quadruple, defining such:

$$\alpha := 10(d+1), \quad \beta := 5(d+1)$$

$$\mathcal{X}_r := 1 - r \frac{d+1}{\alpha} = 1 - \frac{r}{10}$$

Given sets W of points, R of arrows and F of forks, we define U as the set of vertices of the graph induced by the edges of $R \cup F$. The sets W, R and F form an explanation of a spanned slice $\hat{K} = \langle K, v_1, \dots, v_{d+1} \rangle$ of order $\text{order}(\hat{K}) = r$, if and only if:

1. $W \cup \{v_1, \dots, v_{d+1}\} \subset U \subset H_K$ and the graph $(U, R \cup F)$ is connected.
2. All elements of W are leaves.
3. For $m = |W|$ and $s = \mathbf{Size}(K)$ we have $|R| \leq \alpha(m - \mathcal{X}_r) - \beta s$ and $|F| < m - 1$.

Claim 9.2. Every spanned slice $K = \langle K, v_1, \dots, v_{d+1} \rangle$ has an explanation.

Proof. Let $h = T(K) - \min_{u \in H_K} T(u)$. The proof is by induction on h .

1. Base: $h = 0$, meaning T is constant on H_K , so no arrows connect points of H_K . Thus, H_K , as well as $\mathbf{K}$, consists only of a single leaf, namely $\hat{K}$ is a result of a fault, and $r = \text{order}(\mathcal{X}_r) \in \{0, 2\}$. In that case, there is a single vertex $u \in V_{\mathcal{H}}$ such that $U = W = \{u\}$, so $m = 1$, $s = 0$, and $|R|, |F| = 0$. It is easy to check that those values satisfy the requirement.
2. Induction Step: $h \neq 0$ implies $K \neq H_K$. Thus, by claim 8.12, we know that for some k there exist spanned slices $\hat{K}_0, \dots, \hat{K}_k$ and forks $F_1, \dots, F_k$ that satisfy claim 8.12. In particular, for $r < 3$ it holds that:

$$\sum_{i=0}^k \mathbf{Size}(\hat{K}_i) + \sum_{i=1}^k \mathbf{Size}(F_i) \geq \mathbf{Size}(\hat{K})$$

Whereas, for $r = 3$:

$$\sum_{i=0}^k \mathbf{Size}(\hat{K}_i) + \sum_{i=1}^k \mathbf{Size}(F_i) \geq \mathbf{Size}(\hat{K}) + \xi = \mathbf{Size}(\hat{K}) + 2$$

Consider $i \in \{0, \dots, k\}$, and denote $s_i = \mathbf{Size}(\hat{K}_i)$. By the inductive hypothesis and since $T(K_i) = T(K) - 1$, we know that $\hat{K}_i$ has an explanation formed by a set of leaves $\mathbf{W}_i$, a set of arrows $\mathbf{R}_i$, and a set of forks $\mathbf{F}_i$. Let $m_i = |\mathbf{W}_i|$. Thus, the sets $\mathbf{R}_i$ and $\mathbf{F}_i$ have at most $\alpha(m_i - \mathcal{X}_{r-1}) - \beta s_i$ arrows and $m_i - 1$ forks. We define:

- (a) $\mathbf{W} = \cup_{i=0}^k \mathbf{W}_i$, Notice that $\mathbf{W}_i \subset K_i$ and that $T(K_i) = T(K_j)$ for all i, j , thus by Claim 8.8 $\mathbf{W}$ is a union of disjoint set, Hence W contains $m = \sum_{i=0}^k m_i$ leaves.

- (b) $\mathbf{F} = \{F_1, \dots, F_k\} \cup_{i=0}^k \mathbf{F}_i$ with at most $k + \sum_{i=0}^k m_i - 1 = m - 1$ elements.
(c) $\mathbf{R} = \{\{v_i, \text{pre}_i(v_i)\}\} \cup_{i=0}^k \mathbf{R}_i$ with at most

$$\begin{aligned} & d + 1 + \sum_{i=0}^k \alpha(m_i - 1) - \beta s_i \\ & \leq d + 1 + \alpha m - \alpha(k + 1) - \beta(s + \xi - k \cdot \mathbf{Size}(\text{fork})) \\ & = \alpha(m - 1) - \beta s + (\alpha + d + 1 - (k + 1)\alpha - \beta\xi + \beta k \cdot \mathbf{Size}(\text{fork})) \end{aligned}$$

So its enough to show that for $k \geq 0$ the term in the closure is negative, We do that by showing that the slop is negative, and then the value of the term at zero is negative:

$$\begin{aligned} -\alpha + \beta \mathbf{Size}(\text{fork}) &\leq 0 \Rightarrow \mathbf{Size}(\text{fork}) \leq \frac{\alpha}{\beta} \\ d + 1 - \beta\xi &\leq 0 \Rightarrow \frac{d + 1}{\beta} \leq \xi \end{aligned}$$

And those inequalities hold for $\beta = \frac{d+1}{\xi}$ and $\alpha = \beta \mathbf{Size}(\text{fork})$. Finally, by claim 8.12, the graphs induced by $\mathbf{R}_i \cup \mathbf{F}_i$ are connected, thus the graph induced by $\mathbf{R} \cup \mathbf{F}$ is also connected. Therefore, $\mathbf{W}, \mathbf{R}, \mathbf{F}$ form an explanation of K . $\square$

[COMMENT] Consider the proof above, when we run two sequences steps, the first is not shrinking, namely $\xi = 0$, and the second is shrinking $\xi \neq 0$. Each of the proceeding slices $K_0, \dots, K_k$ has its own k_i proceeding slices $K_i^0, \dots, K_i^{k_i}$. We count the number of arrows in K by adding the arrows resulting in those two steps $d + 1 + (k + 1) \cdot (d + 1)$ to the arrows in each of the explanation of the K_i^j slices. So:

$$\begin{aligned} & (k + 2)(d + 1) + \sum_{i,j=0} \alpha(m_i^j - 1) - \beta s_i^j \\ & \leq (k + 2)(d + 1) + \alpha m - \alpha \sum_{i=0}^k k_i - \beta \left(s + \xi - \left(k + \sum_{i=0}^k k_i - 1 \right) \cdot \mathbf{Size}(\text{fork}) \right) \\ & \leq \alpha(m - 1) - \beta s + \left(\alpha + (k + 2)(d + 1) - \alpha \sum_{i=0}^k k_i - \beta\xi + \beta \left(\sum_{i=0}^k k_i \cdot \mathbf{Size}(\text{fork}) \right) \right) \end{aligned}$$

Now, we want to show that the left term, the linear line, is always negative. For this we look for parameters such when $k = 0, \sum k_i = 0$ it results with negative value and that the 'slope' of it is negative. For showing negative slope, we use the fact that $\sum_i k_i \geq k$.

$$\begin{aligned} k = 0 &\Rightarrow \alpha - \beta\xi \leq 0 \Rightarrow \xi \geq \frac{\alpha}{\beta} \\ &\alpha + (k + 2)(d + 1) - \beta\xi - k(\alpha - \beta \cdot \mathbf{Size}(\text{fork})) \\ &\Rightarrow d + 1 - (\alpha - \beta \cdot \mathbf{Size}(\text{fork})) \leq 0 \\ &\Rightarrow \alpha \geq \beta \mathbf{Size}(\text{fork}) + d + 1 \end{aligned}$$

That is not satisfying the inequalities, one way to try to bypass it is treating to $O(d)$ SWEEP RULE operations as a constant depth operation, then $\xi = O(d)$. For that we need the follow property, for every subset of qubits S , there is subset of qubits S' at size at least $(1 - \gamma)|S|$ such that for S to be deviated, all the qubits in S' have to absorb fault. Now suppose that the gate is at depth at most Δ , and the fanning is at most c_f , then we get:

$$\begin{aligned}\Pr[S \in \mathcal{D}] &\leq (c_f^\Delta p)^{|S'|} \leq (c_f^\Delta p)^{(1-\gamma)|S|} \\ \Rightarrow p &\rightarrow (c_f^\Delta p)^{1-\gamma}\end{aligned}$$

A trivial lower bound for $1 - \gamma$ is $\frac{1}{c_f^\Delta}$.

$$\begin{aligned}d + 1 + \sum_{i=0}^k \alpha(m_i - \mathcal{X}_{r-1}) - \beta s_i \\ \leq d + 1 + \alpha m - \mathcal{X}_{r-1} \alpha(k + 1) - \beta(s + \xi - k \cdot \mathbf{Size}(\text{fork})) \\ = \alpha(m - \mathcal{X}_r) - \beta s + ((\mathcal{X}_r - \mathcal{X}_{r-1})\alpha + d + 1 - \mathcal{X}_{r-1}k\alpha - \beta\xi + \beta k \cdot \mathbf{Size}(\text{fork})) \\ - \alpha\mathcal{X}_{r-1} + \beta\mathbf{Size}(\text{fork}) \leq 0 \Rightarrow \mathbf{Size}(\text{fork}) \leq \frac{\alpha}{\beta}\mathcal{X}_{r-1} \\ (\mathcal{X}_r - \mathcal{X}_{r-1})\alpha + d + 1 \leq 0 \Rightarrow \mathcal{X}_r \leq \mathcal{X}_{r-1} - \frac{d+1}{\alpha} \\ \Rightarrow \mathcal{X}_1 \leq \mathcal{X}_0 - \frac{d+1}{\alpha} \\ \Rightarrow \mathcal{X}_2 \leq \mathcal{X}_0 - 2\frac{d+1}{\alpha} \\ (\mathcal{X}_3 - \mathcal{X}_2)\alpha + d + 1 - \beta\xi \leq 0 \Rightarrow \mathcal{X}_3 \leq \mathcal{X}_0 - 3\frac{d+1}{\alpha} + \frac{\beta}{\alpha}\xi \\ \Rightarrow \mathcal{X}_0 \leq \mathcal{X}_0 - 4\frac{d+1}{\alpha} + \frac{\beta}{\alpha}\xi \\ 1 \leq \frac{\alpha}{\beta} \left(1 - 3\frac{d+1}{\alpha}\right)\end{aligned}$$

That holds for $\alpha = d^2, \beta = \frac{1}{2}d^2$,

10 Counting Subtrees.

Claim 10.1. Let $G = (V, E)$ be a $\Delta + 1$ -regular graph, and let k be an integer less than $|E|$. Fix a vertex $r \in V$. The number of subgraphs rooted at r with less than k edges is at most α^k .

Proof. Denote by T_Δ and T_2 the infinity Δ and binary tress. There is an injection between the subtrees of G , rooted at v , with at most k edges, to the subtrees of T_Δ rooted at v , with at most k edges. Moreover, there is an injection from the subtrees of T_Δ rooted at v , with at most k edges to the subtrees of T_2 , rooted at v , with at most $k \log \Delta$ edges.

[COMMENT] In the second map, we think on replacing each of the vertex with $\Delta - 1$ -leaves binary tree, each leaf is associated with a outgoing edge. .

Thus, it's enough to bound the number of of subtrees in T_2 rooted at v , with at most $k \log \Delta$ edges, that number is less than the number of binary tress with $k \log \Delta + 1$ leaves. For exact leaves number we get the $k \log \Delta$ Catalan number, which its limit is:

$$\sim \frac{4^{k \log \Delta}}{(k \log \Delta)^{\frac{3}{2}} \sqrt{\pi}} \leq \frac{\Delta^{2k}}{(k \log \Delta)^{\frac{3}{2}} \sqrt{\pi}}$$

Therefore we can bound by:

$$\sum_{j=1}^k \frac{\Delta^{2j}}{(j \log \Delta)^{\frac{3}{2}} \sqrt{\pi}} \leq \frac{\Delta^{2k}}{\sqrt{\pi} \log^{\frac{3}{2}} \Delta} \sum_{j=1}^k \frac{1}{j^{\frac{3}{2}}} \leq \zeta\left(\frac{3}{2}\right) \cdot \frac{\Delta^{2k}}{\sqrt{\pi} \log^{\frac{3}{2}} \Delta}$$

□

Now, from here we get an upper bound on the number of subgraphs that connected l vertices at the top level. We bound it from above by counting combination of rooted subtrees, such the vertices are their root and the overall edges number is summed up to k . Thus:

$$\leq \zeta\left(\frac{3}{2}\right) \binom{k-1}{l-1} \frac{\Delta^{2k}}{\sqrt{\pi} \log^{\frac{3}{2}} \Delta}$$

11 Tree Separator Lemma

For a tree $T = (V, E)$ and a vertex $v \in V$, we call the subtree obtained by taking the descendants of v the subtree rooted at v .

Claim 11.1. Let $T = (V, E)$ be a tree with $m > 3$ leaves and τ edges, then it has a subtree with $m' \in (\frac{1}{3}m, \frac{2}{3}m)$ leaves and at most $\frac{m'}{m}\tau$ edges.

Proof. Order the vertices according to their heights. Denote by $v \in V$ the lowest vertex for which the subtree rooted at v has at least $\frac{1}{3}m$ leaves. Now, denote by T_1 the subtree obtained through the following iterative process over v 's children:

1. Initialize T_1 as the subtree rooted at v .
2. For a child u of v :
 - (a) Check if erasing from T_1 the subtree rooted at u doesn't decrease the number of leaves below $\frac{1}{3}m$. If so, then we update T_1 to be the result of the erasure.

Clearly, the algorithm is defined such that T_1 has to have at least $\frac{m}{3}$ leaves. Furthermore, since any child of v is a root of a subtree with fewer than $\frac{m}{3}$ leaves (otherwise we get a contradiction to the minimality of v 's height), we have that if T_1 would have more than $\frac{2}{3}m$ leaves, then the subtraction of any of its children would yield a tree with at

least $\frac{2}{3}m - \frac{1}{3}m = \frac{1}{3}m$ leaves. Namely, v has in T_1 a child u such that the subtraction of the subtree rooted at u from T_1 results in a subtree with more than $\frac{1}{3}m$ leaves. But if indeed there were such a child, then the algorithm would erase it, so there is not.

Now let $T_2 = T/T_1 \cup v$, namely the tree induced by the subtraction of T_1 's edges from T . Observe that since the summation of leaves in T_1 and T_2 is the leaves in T , and T_1 has no more than $\frac{2}{3}m$ leaves, and no less than $\frac{1}{3}m$ leaves, it holds that the number of leaves in T_2 is also in the range $(\frac{1}{3}m, \frac{2}{3}m)$. We claim that at least one of the subtrees has fewer than $\frac{m'}{m}\tau$ edges.

To see this, denote by m_1, m_2 and by τ_1, τ_2 the leaves and the edges in T_1, T_2 respectively. Now:

$$\frac{\tau}{m} = \frac{\tau_1 + \tau_2}{m_1 + m_2} = \frac{m_1}{m_1 + m_2} \frac{\tau_1}{m_1} + \frac{m_2}{m_1 + m_2} \frac{\tau_2}{m_2}$$

Namely, $\frac{\tau}{m}$ is a weighted average of $\frac{\tau_1}{m_1}$ and $\frac{\tau_2}{m_2}$, and therefore it is greater than their minimum. Peak the subtree with the minimal quotient. $\square$

By repeating recursively on the above construction, we get the following corollary:

Corollary 11.2. Let $T = (V, E)$ be a tree with m leaves and τ edges. Then, for any $\alpha > 0$, it has a subtree with $m' \in (\frac{1}{3}\alpha m, \frac{2}{3}\alpha m)$ leaves and at most $\frac{m'}{m}\tau$ edges.

Claim 11.3. Let $\gamma, \beta \in \mathbb{R}_{>0}$ and n be a positive integer. For any tree T with $\tau \geq \beta n$ edges and m leaves such that $\tau \leq \gamma m$, there is a subtree T' of T with a number of edges τ' less than $\frac{\beta}{2}n$ and a number of leaves greater than $\frac{\beta}{4\gamma}n$.

Proof. Since the construction of claim 11.1 preserves the ratio of edges to leaves, we have that for a T' obtained using the construction, the number of edges satisfies $\tau' \leq \gamma m'$. So, it's enough to look for the maximal α such that:

$$\gamma m' \leq \gamma \frac{2}{3}\alpha m \leq \frac{\beta}{2}n \Rightarrow \alpha = \frac{3\beta}{4\gamma} \frac{n}{m}$$

Which gives $m' \geq \frac{1}{3}\alpha m = \frac{\beta}{4\gamma}n$. $\square$

Move this definition to the beginning of the paper.

Definition 11.4. We model the qubits on the finite toric with a side length n by the projection:

$$(x_1, x_2, \dots, x_d) \mapsto (x_1 \bmod n, x_2 \bmod n, \dots, x_d \bmod n)$$

Rewrite when we talk about the first time we have deviation clusters of size $\in (\alpha d, d)$. The reason is that having those assumptions means all the droplets in the past shrank; thus, we can use the ratio of edges to time-leaves in the big subgraph.

Claim 11.5. Assume that until step τ , there was no deviation cluster of size greater than d . Let $\hat{K}$ be a spanned slice induced by a deviation of a cluster state at step τ , and let ω be an explanation for $\hat{K}$ such that the number of vertices in ω is greater than βn . Then there is a subgraph (subtree) ω' of ω such that ω' has fewer than $\frac{\beta}{2}n$ edges and at least $\frac{\beta}{4\gamma}n$ leaves.

The connection between the size of the clusters and whether they shrink or not has not been written yet. Moreover, it seems that there might be confusion since two disjoint clusters might be time-connected. It seems that might be a problem.

Proof. Let ω be an explanation of $\hat{K}$ with more than βn vertices. Since there is no deviation cluster of size greater than d until step τ , we have that every vertex of ω is a spanned slice induced by a boundary of a deviation cluster of size at most d . Thus, ω satisfies the shrink property in Claim 17.6, and by Claim 9.2, the ratio of edges to leaves of ω is at most γ . Observe that a spanning tree of ω has a ratio of edges to leaves that is at most ω 's ratio, and the number of edges equals ω 's vertices minus 1. Therefore, by applying Claim 11.3 on ω 's spanning tree, ω has a subtree with less than $\frac{\beta}{2}n$ edges and at least $\frac{\beta}{4\gamma}n$ leaves. $\square$

State a claim, and notice that in the new version we are not only interested in trees whose root is a single vertex.

Proof. Now, since any two connected vertices are at a space-distance of at most 1, it follows that all the leaves in the subtree are at a space-distance of at most βn from each other. Thus, if $\beta < 1$, no two of them have the same space-coordinate modulo n . And therefore the probability to have such subtree is at most $p^{\frac{\beta}{4\gamma}n}$.

Because any ω with more than βn vertices implies the existence of a subtree with size less than $\frac{\beta}{2}n$, it's enough to bound the probability of having such a subtree. Considering that the trees are invariant to shifting by $n \cdot \mathbf{1}_i$, we have that it's sufficient to show that there are no such trees which are rooted in the finite cube $\mathbb{Z}_n^d \times \mathbb{Z}_t$. Finally, we use claim 10.1 to bound the number of such trees given a fixed root. Overall, we get:

$$\Pr[\text{No } \omega \text{ s with more than } \beta n \text{ vertices}] \leq n^{dt} \left(\xi^2 p^{\frac{\beta}{4\gamma}} \right)^n$$

$\square$

12 Tanner - History Connected Equivalency

Imagine the following process, we apply t noisy iterations of the SWEEP rule, and then a single ideal (without noise) iteration. Denote by ξ a deviated configuration (can be either bits or checks), and by ξ' the result of applying an ideal SWEEP rule on ξ . Notice that if ξ is connected in the Tanner graph then ξ' is connected in the

History graph [COMMENT] Require proof, For the classical case enumerate all the options can be done by hand. Thus:

$$\begin{aligned}
\Pr[\xi \text{ connected in Tanner}] &\leq \Pr[\xi' \text{ connected in Time} \mid \text{no noise at the } t+1 \text{ iteration}] \\
&\leq \sum_m^\infty |\{\text{explanation to } \xi' \text{ over } m \text{ leaves restricted to no noise at the final iteration}\}| p^{\frac{m}{\Delta}} \\
&\leq \sum_m^\infty |\{\text{explanation to } \xi' \text{ over } m \text{ leaves}\}| p^{\frac{m}{\Delta}} \\
&\leq \Pr[\xi' \text{ connected in Time} - \text{Upper Bound}] \leq q^{\text{Span}(\xi')} \leq q^{\text{Span}(\xi)-1}
\end{aligned}$$

12.1 Another Idea.

Reconsider the construction of the explanation. We could add at the top an imaginary layer in which all the vertices at time t are connected by arrows to all the vertices (at time $t+1$) that are adjacent to them in the Tanner graph. For the last layer, we might find that the span of a slice expands instead of shrinking. Namely, claim 8.12 for the final slices would change to:

$$\sum_{i=0}^k \text{Size}(\hat{K}_i) + \sum_{i=1}^k \text{Size}(F_i) \geq \text{Size}(\hat{K}) - \xi$$

Thus, following the computation at the induction stage in claim 9.2, the number of arrows is bounded by:

$$\begin{aligned}
&\alpha(m-1) - \beta s + d + 1 + \alpha + \beta \xi \\
&\alpha(m-1) - \beta s + O(1)
\end{aligned}$$

13 Intermediate Step Lemma

[COMMENT] The idea of this step is also to bound the case that all the bits are flipped at once, So checks are satisfied (and the argue above is not applied directly).

Definition 13.1 (Connected Cluster With Span s). Let $G = (B, C, E)$ be a Tanner graph of a code, and let $f: G \rightarrow \mathbb{R}^{d+1}$ be an embedding of the graph into $\mathbb{R}^{d+1}$. For $s \in \mathbb{R}$, we say that $X \subset B$ is a connected cluster of G with span s regarding f if X is connected in G (where two bits are connected if there is a check that connects both of them in E) and $f(X) \subset \mathbb{R}^{d+1}$ has $\text{Size}(f(X)) \leq s$. When it's clear from the context which G and f are, we will call X in brief a connected cluster with span s .

Definition 13.2. The model, computation, and noise are tossed, and then a correction is made.

Claim 13.3. Let ξ be the event in which, at the **beginning** of time t , the error contains a connected cluster with a span greater than $\frac{d}{4}$, and for $i \in [t]$, let σ_i be the event in which, at the **end** of time i , there is a connected cluster with a span of at least $\sqrt{d}$ that belongs to the error. Then:

$$\Pr[\xi] \leq t\Pr[\sigma_1] + p^{\frac{1}{8}\sqrt{d}}$$

Proof. By the law of total probability it holds:

$$\Pr[\xi] = \Pr[\xi \cap \bigcup_i^t \sigma_i] + \Pr[\xi \cap \bigcap_i^t \bar{\sigma}_i]$$

First, let us bound from above the probability of the right intersection, In that case, at the **end** of time $t - 1$, the error consists of clusters each at span less than $\sqrt{d}$. Let m be the number the clusters and denote them by $X_1, \dots, X_m$. Now since the clusters are disjoint, the sum of their **Size**(X_i) is lower than the total number of the bits, namely:

$$\begin{aligned} \sum_{i=1}^m \text{Size}(X_i) &\leq n \Rightarrow m \leq \frac{n}{\sqrt{d}} \\ p \frac{n}{\sqrt{d}} &\leq \sqrt{d} \end{aligned}$$

, Thus, . So denote by τ the number of bits that require to be flipped in order to merge the flipped connected component to a one with a span greater than $d/4$, So τ has to be at least:

$$\tau (\sqrt{d} + 1) \geq \frac{1}{4}d \Rightarrow \tau \geq \frac{1}{8}\sqrt{d}$$

Therefore:

$$\begin{aligned} \Pr[\xi \cap \bigcap_i^t \bar{\sigma}_i] &\leq \sum_{\xi^{(t-1)}} \Pr[\xi | \xi^{(t-1)}] \Pr[\xi^{(t-1)} \cap \bigcap_i^t \bar{\sigma}_i] \\ &\leq \sum_{\xi^{(t-1)}} p^{\frac{1}{8}\sqrt{d}} \Pr[\xi^{(t-1)} \cap \bigcap_i^t \bar{\sigma}_i] \leq p^{\frac{1}{8}\sqrt{d}} \end{aligned}$$

$$\begin{aligned} \Pr[\xi] &= \Pr[\xi \cap \bigcup_i^t \sigma_i] + \Pr[\xi \cap \bigcap_i^t \bar{\sigma}_i] \\ &\leq t\Pr[\sigma_1] + p^{\frac{1}{8}\sqrt{d}} \end{aligned}$$

□

Definition 13.4. The atomic events are the sets of space-time locations in which faults occur. A run ω is the sequence of states (assignment of all the bits) induced by an atomic event (and the transition function of the machine). We say that a run ω closes a non-trivial cycle if it contains a state in which there is a connected cluster A such that its boundary is not a co-boundary.

Define the bits-cluster graphs $G = (V, E)$ such that the vertices are connected bit-clusters at different times $\{A_i^t\}$, and there is an edge between A_i^t and A_j^{t-1} only if $\Pi_x A_i^t \cap \Pi_x A_j^{t-1} \neq \emptyset$. Given a state at time $t - 1$ such that no connected component in G forms a closed non-trivial circle, consider $\tilde{A}_i^t$ constructed as follows: perform the application of either noise or Toom's correction in an arbitrary order and stop right before $\tilde{A}_i^t$ forms a closed non-trivial circle.

Claim 13.5. There is $\phi \in \text{Aut}(\mathbb{T}_n^d)$ for which $\text{Size}(\phi(\tilde{A}_i^t)) \geq n - 1$.

Claim 13.6. Consider a running in which no non-trivial circle was closed, then there is $\psi: \mathbb{T}_n^d \rightarrow \mathbb{Z}_{2n}^d$ such:

1. For any slice K , $\psi(K)$ is connected in $\mathbb{Z}_{2n}^d$.
2. For any t , denote by S_t the state at time t . Then, for any $t > 1$, we have that $\psi(S_t)$ is the image of the application of the Tooms rule over $\psi(S_{t-1})$.

Proof. We prove it by induction on time t .

1. Base case, $t = 1$. We will construct ψ in the following iterative manner. First, let ψ be the identity. Then, while there are slices that are not connected on $\mathbb{Z}_{2n}^d$, do the following: Let K be a slice that is not connected on $\mathbb{Z}_{2n}^d$. Then it can be decomposed into a disjoint union $K = K_1 \cup K_2$ such that K_1 and K_2 are connected on $\mathbb{Z}_{2n}^d$. Assume, without loss of generality, that for a coordinate $x \in [d]$, it holds that for any $a \in K_1$ and $b \in K_2$, we have $b_x - a_x > 1$.

Observe that there has to be at least one such coordinate; otherwise, it is a contradiction to K_1 and K_2 not being connected in $\mathbb{Z}_{2n}^d$. Then define:

$$\psi(z) \leftarrow \begin{cases} \psi(z) & z \cap K_1 = \emptyset \\ z + n\mathbf{1}_x & \text{else} \end{cases}$$

Repeat the above for other coordinates if $\psi(K)$ is still not connected in $\mathbb{Z}_{2n}^d$. The proof that $\psi(K)$ has to be connected (that the process ends) is omitted.

2. Induction step. We repeat the construction in the base case. To prove the second property, we show that ψ maps (un)satisfied edges to (un)satisfied edges. [\[COMMENT\] That is wrong.](#)

□

Claim 13.7. Denote by ω running of the above type, then there is a running ω' on $\mathbb{Z}_\infty^d$ such that ω and ω' agree.

Claim 13.8. Consider a t -steps running, conditined on the event that no trivial-cycle was closed in the $t - 1$ first steps, the probability that a non-trivial cycle will be closed at the t th step is less than:

$$\Pr[\Lambda^t | \bigcap_{t' < t} \bar{\Lambda}^{t'}] < n^{2d} \sum_{i=n-1}^{\infty} p^i \mathcal{J}_i \leq n^{2d} q^{n-1}$$

Proof. Consider the event in which there is a droplet that at time t closes a non-trivial cycle. Let $\tilde{A}$ be the subset of that droplet obtained by the process in claim 13.5. By the claim, $\tilde{A}$ has a boundary σ and two vertices on that boundary $M = \{u, v\}$ such that $\text{Size}(M) > n - 1$, up to isomorphism.

Now, since no slice in the investigation of M closes a non-trivial cycle then by claim 13.6 .. so by claim 13.7 .. $\square$

14 Configuration up to $C_z^\perp$

15 Drafts:

In addition since, in a tree, the number of leaves is at least greater than the maximal degree, then we have that each J_j intersects with at most $d + 1$ elements.

$$L + \Delta_{\max} + 2(n - L - 1) \leq \sum_{v \in V} \deg(v) = 2(n - 1)$$

$$\Rightarrow L \geq \Delta_{\max}$$

16 Decoder Phase

In the general picture, I would like to use the 4D toric code to encode in any target code. We use the original fault tolerance construction (the concatenation construction) to encode into and decode from the 4D toric code. So, for the decoding phase, we need to have a decoder which, in the ideal setting, runs in less than logarithmic time.

What we know so far:

1. We can assume that two unsatisfied checks are belong to the same history with probability less than $\sim q^{|u-v|}$.
2. There is a $\log^2(n)$ depth algorithm to find the perfect/max matching in a general graph. From first check, the known algorithms to deal with the weight matching are based on LP.
3. Applying Toom's/Sweep rule will require at least linear depth.
4. Having an algorithm that succeeds with constant probability $\frac{1}{2} + \epsilon$ might be enough (Anyway the qubit at the result of the decoding is going to absorb p -noise).

17 Appendix

Do something with that. And clearly Stab_v is collection of stabilizers since for any

$S' \subset S$ such $|S'| = d' + 1$ we have:

$$\begin{aligned}
\partial(\partial\theta_{\mathbf{v}, S'}) &= \partial\left(\sum_{g \in S'} \theta_{\mathbf{v}, S'/g} + \sum_{g \in S'} \theta_{\mathbf{g}\mathbf{v}, S'/g}\right) \\
&= \sum_{\substack{g', g \in S' \\ g' \neq g}} (\theta_{\mathbf{v}, S'/\{\mathbf{g}, \mathbf{g}'\}} + \theta_{\mathbf{g}'\mathbf{v}, S'/\{\mathbf{g}, \mathbf{g}'\}} + \theta_{\mathbf{g}\mathbf{v}, S'/\{\mathbf{g}, \mathbf{g}'\}} + \theta_{\mathbf{g}'\mathbf{g}\mathbf{v}, S'/\{\mathbf{g}, \mathbf{g}'\}}) \\
&= 0
\end{aligned}$$

Where we used that the term in the closure is invariant to sweeping between g and g' , and therefore any term in the sum appears twice.

Definition 17.1. Let z be a connected assignment, and let Γ be the set of vertices adjacent to unsatisfied checks induced by z . For a generator $g \in S$, we say that v is a g -pole if $v \in \Gamma$ and $gv \notin \Gamma$. Furthermore, if there is a vertex $v \in \Gamma$ such that for any $g \in S$, we have $g^{-1}v \notin \Gamma$, then we say that v is a local minimum of z .

Let $x_1, \dots, x_k \subset \{\pm\}^k$ and let $p = \prod_i^k g_i^{x_i} v$ we say that p is a forward path in z if it holds that:

1. Any partial product of the first j elements $\prod_i^j g_i^{x_i} v$ is a vertex in z .
2. For any $g \in S$, we have that the sign of the sum of the exponents of the powers of the elements in the product associated with g is positive. Namely:

$$\sum_{i: g_i = g} x_i > 0$$

Similarly, we say that p is a backward path if the above holds, but the sign in the second condition is either negative or zero. The infimums of z are defined to be the set of vertices $U \subset z$ such that for any $u \in U$ and $v \in z$, other vertices in z , there is a forward path starting at u and ending at v .

17.1 Alternative Proof For $d' = 2$.

Definition 17.2. We denote by ϕ^t the subsets of bits which are flipped as a result of applying the Toom's rule at time t , and by ϕ_v the bits which are flipped by vertex v . We say that u is added to Γ^{t+1} by v at time t , if $u \in \Gamma^{t+1}$ and $u \in V(\phi_v)$. When it clear from the context, we will omit the time, and say that u is added by v .

Claim 17.3. Let $v \in \Gamma^t$ for any u that is added by v , and for any $x \in [d+1]$, we have $L_x(u) \leq L_x(v) + 1$.

Proof. By definition, any u that is added by v is of the form $v + \sum \alpha_j 1_j$, where $\alpha_j \in \{0, 1\}$ and $\sum \alpha_j \leq 2$. Thus, for any $x \in [d]$:

$$L_x(u) = \langle 1_x, v + \sum \alpha_j 1_j \rangle = \langle 1_x, v \rangle + \alpha_x \leq L_x(v) + 1$$

For $x = d + 1$ we have:

$$L_x(u) = \langle -\mathbf{1}, v + \sum \alpha_j 1_j \rangle = \langle -\mathbf{1}, v \rangle - \sum \alpha_j \leq L_x(v)$$

□

Claim 17.4. Let $v \in \Gamma^t$ such that $L_{d+1}(v)$ is a (local) maxima in Γ^t then for any u which is added by v we have $L_{d+1}(u) \leq L_{d+1}(v) - 1$.

Proof. We will prove that $v \notin \Gamma^{t+1}$, and then by repeating on claim 17.3 where $\sum \alpha_j \geq 1$, we get the desired result. Assume, toward contradiction, that $v \in \Gamma^{t+1}$, and denote by w the vertex that adds v . If $w \neq v$, then there is at least one coordinate i for which $w_i < v_i$, thus $L_{d+1}(w) > L_{d+1}(v)$, in contradiction to $L_{d+1}(v)$ being a maximum in Γ^t . In the other case, where v adds itself, it follows that σ_v can't be decomposed into pairs of unsatisfied checks; namely, $|\sigma_v|$ is odd. We show by induction that impossible, namely that $|\sigma_v|$ is always even. The induction is over the size of the plaquettes subset that induce the boundary σ .

1. Base. Consider a boundary that induced by a single plaquette, in that case it obvious that $|\sigma_v|$ is either zero (where v is not belong to the plaquette) or 2.
2. Assume for any boundary that induced by at most k plaquettes, and consider a boundary σ which is induced by $k + 1$ plaquettes, name this subset F and observes that F can be decomposed to a sum of two disjoint subsets A and B , each with less than $k + 1$ plaquettes, Now:

$$\begin{aligned} \sigma_v &= (\partial A)_v \cup (\partial B)_v \\ \Rightarrow |\sigma_v| &= |(\partial A)_v| + |(\partial B)_v| - 2|(\partial A)_v \cap (\partial B)_v| = \text{even} \end{aligned}$$

Where we used the induction assumption, which tells that $|(\partial A)_v|$ and $|(\partial B)_v|$ are even numbers.

□

Claim 17.5. Let $x \in [d]$ and let $v \in \Gamma^t$ such that $L_x(v)$ is a (local) maxima in Γ^t then for any u which is added by v we have $L_x(u) \leq L_x(v)$.

Proof. Since $L_x(v)$ is maximal, v doesn't see edges that goes from it at the positive x -direction, otherwise denote it by e and denote by w the second vertex at the end of e , w has an x -coordinate which is greater than v 's x -coordinate by 1 and in addition $w \in \Gamma^t$ (since it lays on the support of e). Therefore $L_x(w) > L_x(v)$ in contradiction to $L_x(v)$ be maximal in Γ^t . Hence, any plaquette that is being flipped by v lies on edges associated with 1_j such that $j \neq x$. Thus, any vertex u that is added by v has the form $v + \sum_{j \in [d] \setminus \{x\}} \alpha_j 1_j$, where $\alpha_j \in \{0, 1\}$ and $\sum \alpha_j \leq 2$. So:

$$L_x(u) = \langle 1_x, v + \sum_{j \in [d] \setminus \{x\}} \alpha_j 1_j \rangle = \langle 1_x, v \rangle = L_x(v)$$

□

Claim 17.6. Let Γ^t be the vertices in the support of the domain wall σ^t . Denote $v_i = \arg \max_{v \in \Gamma^t} L_i(v)$. Then for any $i \in [d]$, one can find $u \in \Gamma^{t-1}$ such that u adds v_i and $L_i(u) \geq L_i(v_i)$. In addition, one can find $u_{d+1} \in \Gamma^{t-1}$ such that u_{d+1} adds v_{d+1} and $L_{d+1}(u_{d+1}) = L_{d+1}(v_{d+1}) + 1$.

Proof. Just take u_i to be any vertex that adds v_i . By claims claim 17.3, claim 17.4, and claim 17.5, we get the claim. $\square$

[COMMENT] $V(\phi_v)$ was not defined. $v \leq w$ also not defined.

Definition 17.7 (σ_1 Precedes σ_0). We say that σ_1 precedes σ_0 if, for any $v \in \sigma_1$, it holds that $\phi(v) \subset \sigma_0$.

Definition 17.8 (Explanation.). Let σ_0 be a boundary at time t , and let $\sigma_1, \dots, \sigma_k$ be boundaries at time $t-1$, such that for $i \in [k]$, it holds that σ_i precedes σ_0 . Then the explanation for σ_0 is the subgraph induced by the vertices of $\sigma_0, \sigma_1, \dots, \sigma_k$, the edges $\{u, v\} \in \sigma_i$, and the arrows $\{v, u\}$ for $v \in \cup \sigma_i$ and $u \in \phi(v)$.

The number of edges above is not lineary bounded by the leaves. Yet it also seems not minimal. So what we would like to do is to keep the poles, then to connect the islands by fork. So what is it a fork?

Definition 17.9 (Fork). Let u, v be such that $u \in \sigma_1$ and $v \in \sigma_2 \neq \sigma_1$. If there exist $w \in \phi(v)$ and $z \in \phi(u)$ such that $\{w, z\} \in \mathcal{F}_1$, then there is a fork between v and u .

Proof. Idea: Suppose not, then $\sigma_0 = \phi(\sigma_1) \cup \phi(\sigma_2)$ is not connected. $\square$

Claim 17.10. Assume that $\sigma_1, \sigma_2 \rightarrow \sigma_0$, then there is a fork between σ_1 and σ_2 . So if we keep to extent the history in a recursive manner, then we get a connected graph.

18 Garbage

18.1 Drafts-Garbage

[COMMENT] Regarding the Claim 7.5, The number of restriction in ∂^+ case was counted in negligently. In addition, we should show that for the ∂^- case, restrictions of the form $\theta_{\mathbf{g}_2 \mathbf{g}_1 \mathbf{v}, \mathbf{S}}$ are degenerate. In the bottom line it's look like:

$$\partial^+ \theta_{\mathbf{g}_2 \mathbf{g}_1 \mathbf{v}, \mathbf{S}} = \partial^+ \theta_{\mathbf{g}_1 \mathbf{v}, (\mathbf{S}/\mathbf{g}_3) \cup \mathbf{g}_2} + \partial^+ \theta_{\mathbf{g}_2 \mathbf{v}, (\mathbf{S}/\mathbf{g}_3) \cup \mathbf{g}_1}$$

Lets develop it, consider a restriction rooted at $g_2 g_1 v$, and let's look on it's boundary:

$$\partial^+ \theta_{\mathbf{g}_2 \mathbf{g}_1 \mathbf{v}, \mathbf{S}'} = \sum_{g \in \mathbf{S}/\mathbf{S}'} \theta_{\mathbf{g}_2 \mathbf{g}_1 \mathbf{v}, \mathbf{S}' \cup \mathbf{g}} + \sum_{g \in \mathbf{S}/\mathbf{S}'} \theta_{\mathbf{g}^{-1} \mathbf{g}_2 \mathbf{g}_1 \mathbf{v}, \mathbf{S}' \cup \mathbf{g}}$$

Now, any term in the left closure is a bit beyond the scope of the bits in Stab_v^- , thus, we are zeroing them out. The bits, on the right closers are in the scope, only if $g \in \{g_1, g_2\}$. Thus, in total the check is equivalence to:

$$\begin{aligned} \partial^+ \theta_{\mathbf{g}_2 \mathbf{g}_1 \mathbf{v}, \mathbf{S}'} &= \theta_{\mathbf{g}_1 \mathbf{v}, \mathbf{S}' \cup \mathbf{g}_2} \cdot \mathbf{1}_{g_2 \notin \mathbf{S}'} \\ &\quad + \theta_{\mathbf{g}_2 \mathbf{v}, \mathbf{S}' \cup \mathbf{g}_1} \cdot \mathbf{1}_{g_1 \notin \mathbf{S}'} \end{aligned}$$

$$\begin{aligned}
\partial^+ \theta_{\mathbf{g}_1 \mathbf{v}, (\mathbf{S}'/\mathbf{g}_3) \cup \mathbf{g}_2} &= \sum_{g \in S/S'} \theta_{\mathbf{g}_1 \mathbf{v}, (\mathbf{S}'/\mathbf{g}_3) \cup \mathbf{g}_2 \cup g} + \sum_{g \in S/S'} \theta_{\mathbf{g}^{-1} \mathbf{g}_1 \mathbf{v}, (\mathbf{S}'/\mathbf{g}_3) \cup \mathbf{g}_2 \cup g} \\
\partial^+ \theta_{\mathbf{g}_2 \mathbf{v}, (\mathbf{S}/\mathbf{g}_3) \cup \mathbf{g}_1} &= \sum_{g \in S/S'} \theta_{\mathbf{g}_2 \mathbf{v}, (\mathbf{S}'/\mathbf{g}_3) \cup \mathbf{g}_1 \cup g} + \sum_{g \in S/S'} \theta_{\mathbf{g}^{-1} \mathbf{g}_2 \mathbf{v}, (\mathbf{S}'/\mathbf{g}_3) \cup \mathbf{g}_1 \cup g} \\
\partial^+ \theta_{\mathbf{g}_1 \mathbf{v}, (\mathbf{S}'/\mathbf{g}_3) \cup \mathbf{g}_2} &= \left(\sum_{g \in S/S'} \theta_{\mathbf{g}_1 \mathbf{v}, (\mathbf{S}'/\mathbf{g}_3) \cup \mathbf{g}_2 \cup g} \right) + \theta_{\mathbf{v}, (\mathbf{S}'/\mathbf{g}_3) \cup \{\mathbf{g}_2, \mathbf{g}_1\}} \\
\partial^+ \theta_{\mathbf{g}_2 \mathbf{v}, (\mathbf{S}/\mathbf{g}_3) \cup \mathbf{g}_1} &= \left(\sum_{g \in S/S'} \theta_{\mathbf{g}_2 \mathbf{v}, (\mathbf{S}'/\mathbf{g}_3) \cup \mathbf{g}_1 \cup g} \right) + \theta_{\mathbf{v}, (\mathbf{S}'/\mathbf{g}_3) \cup \{\mathbf{g}_1, \mathbf{g}_2\}}
\end{aligned}$$

Other way:

$$\begin{aligned}
\partial^+ \sum_{g' \in S/S'} \theta_{\mathbf{g}' \mathbf{v}, \mathbf{S}'} &= \sum_{g' \neq g \in S/S'} \theta_{\mathbf{g}' \mathbf{v}, \mathbf{S}' \cup g} + \sum_{g \in S/S'} \theta_{\mathbf{v}, \mathbf{S}' \cup g} \\
&= \sum_{g' \neq g \in S/S'} \theta_{\mathbf{g}' \mathbf{v}, \mathbf{S} \cup g} + \partial^+ \theta_{\mathbf{v}, \mathbf{S}'}
\end{aligned}$$

Where we use the fact that if $g = g'$ then the bit is beyond the scope. In addition:

$$\begin{aligned}
\partial^+ \sum_{g' < g \in S/S'} \theta_{\mathbf{g}' \mathbf{g} \mathbf{v}, \mathbf{S}'} &= \sum_{g' < g \in S/S'} \theta_{\mathbf{g}' \mathbf{v}, \mathbf{S}' \cup g} + \theta_{\mathbf{g} \mathbf{v}, \mathbf{S}' \cup g'} \\
&= \sum_{g' \neq g \in S/S'} \theta_{\mathbf{g}' \mathbf{v}, \mathbf{S}' \cup g} \\
&= \partial^+ \sum_{g' \in S/S'} \theta_{\mathbf{g}' \mathbf{v}, \mathbf{S}'} + \partial^+ \theta_{\mathbf{v}, \mathbf{S}'}
\end{aligned}$$

Thus, we find more $\binom{d}{d'-1}$ restriction over the checks. [We need to show that those restrictions are independent]. In addition:

$$\partial^+ \sum_{S'} \sum_{g' < g \in S/S'} \theta_{\mathbf{g}' \mathbf{g} \mathbf{v}, \mathbf{S}'} = 0$$

$$\partial^+ \theta_{\mathbf{g}_2 \mathbf{g}_1 \mathbf{v}, \mathbf{S}} = \partial^+ \theta_{\mathbf{g}_1 \mathbf{v}, (\mathbf{S}/\mathbf{g}_3) \cup \mathbf{g}_2} + \partial^+ \theta_{\mathbf{g}_2 \mathbf{v}, (\mathbf{S}/\mathbf{g}_3) \cup \mathbf{g}_1}$$

I don't succes to advance the idea more than that, So in the end I believe that I will do is to say that the diminsion of Stab_v^- is at least $\binom{d}{d'+1}$. And by fixing coordinates we add more $\binom{d}{d'}$ restriction. So the dimension becomes:

$$\begin{aligned}
&\binom{d}{d'+1} - \binom{d}{d'} \\
&\binom{d}{d'+1} - \binom{d}{d'} + \binom{d}{d'-1}
\end{aligned}$$

Note that since we talk on mapping checks $\rightarrow$ bits, we use the ∂^+ . Eventually, we need to use the poicare duality $H^{d'} \rightarrow H_{d-d'}$, the duality also should be used to define the SWEEP rule to C_Z . I didn't successes to do it other way.

A wrong proof of Claim 7.5

Proof. We will show that codes obtained from Stab_v by forcing the faces rooted at v to $z|_v$ have non-zero dimensions. Then it follows that any code word of $\mathcal{C}_v$ could be extend to codewords in Stab_v . The number of bits in both Stab_v is:

$$\binom{d}{d'} + d \binom{d-1}{d'}$$

The numbers of checks for $\text{Stab}_\pm$ are:

$$\binom{d}{d' \pm 1} + d \binom{d-1}{d' \pm 1}$$

The restriction to $z|_v$ sets the bits rooted at v and satisfies the checks that are rooted at v , so the checks of the form $\theta_{\mathbf{v}, \mathbf{S}'}$ for $|S'| = d' \pm 1$ are degenerate, and the dimension that remains is:

$$\dim \geq d \binom{d-1}{d'} - d \binom{d-1}{d' \pm 1}$$

In the case that $d' = (d-1)/2$, then for any $x \in [d-1]$ we have $\binom{d-1}{d'} > \binom{d-1}{x}$ therefore:

$$\begin{aligned} \dim &\geq d \binom{d-1}{d/2} - d \binom{d-1}{d/2 \pm 1} \\ &> d \end{aligned}$$

So there is at least 2^d stabilizers $w \in \text{Stab}_v$ that agree with z . □

Proof.

$$L_{d+1}(u) \leq \min_{v \in \partial z} L_{d+1}(v)$$

Observes that if the *termination-condition* holds then any local maximum of L_{d+1} in ∂z is also a local maximum of L_{d+1} in $\tilde{z}$ and we finish. Consider the other case, namely there is at least one vertex $u \in \tilde{z}/\partial z$ such that u is a local maximum for L_{d+1} and

$$L_{d+1}(u) > \min_{v \in \partial z} L_{d+1}(v)$$

In that case, we peak such u , and looking for stabilizer w such $u \notin \tilde{z} + w$ by repeating over the above procedure where u plays the role of v . We keep repeating on the procedure until the *termination-condition* holds.

We argue that the process indeed terminates. Consider a vertex u such that u is a local maximum of L_{d+1} in $\tilde{z}$ and u is not a local maximum of L_{d+1} at z . Put differently, u becomes a local maximum as a result of adding w to z . Thus, u has to belong to $w|_v \setminus v$, which implies that there is a non-empty subset of generators $I \subset S$ such that $u = (\prod_{g \in I} g)v$, and therefore $L_{d+1}(u) \leq -1 + L_{d+1}(v)$.

Hence, since z is finite.

We repeat the process while possible; namely, we stop when all the local maxima of L_{d+1} are in ∂ . □

We prove that $\dim \mathcal{C}_v = \dim \text{Stab}_v$ below, for now assume it is correct. We continue by using the fact that if, for a local maximum v in ∂ , it holds that $z|_v$ is rooted at v , then the decoder will find bits rooted at v such that their ∂ agrees with ∂ .

To generalize, one might think that it has to complete the local view on v such that all the checks are satisfied. For C_X , it sums up to show that the following dimension is positive. Yet the trick fails for C_Z , where we need to think of a different idea.

$$d \binom{d-1}{d'} - \binom{d}{d'-1} - d \binom{d-1}{d'-1}$$

Claim 18.1. Let z an assignment rooted at vertex v , such $\partial z|_{v+} = 0$. Then there is a stabilizer w , rooted at v , such that $w|_{v+} = z$.

Proof. Denote by F the faces in z , and by J the collection of generators that support F . We claim that $|J| \geq d' + 1$.

Clearly, $|J| \geq d'$, since each face is defined by a root vertex and d' generators. Yet, observe that if $|J|$ equals exactly d' , then z contains a single d' -face, denote it by f . Then, any check associated with a $d' - 1$ face of f is unsatisfied, in particular the $d' - 1$ faces that are rooted at v , in contradiction to $\partial z|_{v+} = 0$.

Now, one can choose $J' \subset J$ of size $d' + 1$, and define the stabilizer $w = \theta_{v,J'}$. □

Claim 18.2. Suppose $v \in z$ and for any $g \in S$, we have $g^{-1}v \notin z$, then there is $\theta_{v,J} \in \partial z$.

Proof.

$$\partial z = \sum_{S'} \theta_{v,S'} + ..$$

□

Claim 18.3. Let z be a connected assignment on the d' -faces at finite size, and let Γ be the set of vertices which are adjacent to unsatisfied checks induced by z . Let v be a vertex in $\mathcal{G}_\infty^d$ such that v is a strong global maximum of L_{d+1} , at Γ . Then there is a rooted assignment $\tilde{z}$ at v such that $z_v = \tilde{z}_v$.

Proof. Let $\Theta = \theta_{v_1,S'_1}, \theta_{v_2,S'_2}, .. \theta_{v_k,S'_k}$ be the faces forms z .

Since z is finite, it has a maximum for L_{d+1} . Denote by u the vertex that gets the maximum, and denote by $\theta_{u,J_1}, \dots, \theta_{u,J_l}$ the d' -faces in z that are rooted at u . We claim that $u = v$.

Suppose not, namely $u \notin \Gamma$. Then it means that any $d' - 1$ face containing u is satisfied. Observe that $\sum \theta_{u,J_i} \neq 0$; otherwise, it would contradict the non linearity independence of $\theta_{v_1,S'_1}, \theta_{v_2,S'_2}, .. \theta_{v_k,S'_k}$. So, by Claim 18.2 there exists a $d' - 1$ face rooted at u , contained in $\sum \theta_{u,J_i}$. Let us denote it by $\theta_{u,J'}$.

Since $\theta_{u,J'}$ is satisfied, it has to be adjacent to a turned bit, namely a d' face in z , not in $\theta_{u,J_1}, \dots, \theta_{u,J_l}$. Denote that d' face by f . Yet, since $\theta_{u,J'}$ is rooted at u , it contains u , and therefore f contains u . Now, if f is rooted at u , then it is

a contradiction that $\theta_{u,J_1}, \dots, \theta_{u,J_l}$ are all the d' faces in z rooted at u . If f is not rooted at u , denote its root by w , so there is $g \in S$ such that $gw = u$, and $w \in z$, in contradiction to u being maximum for L_{d+1} .

So, we find that v is also the maximum of L_{d+1} in z . Therefore, we have that $\tilde{z} = \sum \theta_{u,J_i}$ zeros out z_v . $\square$

Claim 18.4. Let z be a connected assignment on the d' -faces at finite size, and let the collection $\Theta = \theta_{v_1,S'_1}, \theta_{v_2,S'_2}, \dots, \theta_{v_k,S'_k}$ be the decomposition forms z . Denote by Γ the set of vertices adjacent to unsatisfied check induced by z . Then Θ is acyclic, if and only if z has at least one Infimum.

Proof. First, suppose that z has at least one Infimum. Let u be an infimum in z . Second, suppose that Θ is acyclic. $\square$

Claim 18.5. Suppose that $d' > 1$ and a vertex $v \in \mathcal{G}_\infty^d$ is also adjacent to the domain wall, namely $v \in \Gamma$. It holds that v cannot be simultaneously both a (strong) local max of L_{d+1} and a local min of L_{d+1} .

Proof. Let $v \in \Gamma$ be a local maximum point of L_{d+1} , It means that:

1. The vertex v is adjacent to a d'^{-1} -face which is unsatisfied.
2. That face is rooted in v ; otherwise, the root of the face is a vertex in Γ with a higher value of L_{d+1} , in contradiction to v being a local maximum.

Thus, there is a subset of generators $S' \subset S$, of size $d' - 1 > 0$, such that the unsatisfied check set is on the face $\theta_{v,S'}$. Given $d' > 1$, we have that S' contains at least one more element besides the empty set, denoted by g . Now, consider the vertex gv . Since gv is adjacent to $\theta_{v,S'}$, then $gv \in \Gamma$. On the other hand, on the infinite toric $\mathcal{G}_\infty^d$, moving along the grid generator decreases L_{d+1} by one, so $L_{d+1}(gv) = L_{d+1}(v) - 1$. In other words, gv is a neighbor of v with a lower value of L_{d+1} , and thus v cannot be a local minimum of L_{d+1} . $\square$

Remark 18.6. That claim alone gives a decoder in the clean setting. Write a remark indicating that when $d' = 1$, meaning qubits are on the edges (2D toric), the claim is not true.

For $d = 4, d' = 2$. For $i, j \in [d]$, such that $i \neq j$, denote the bit rooted at v and spanned by generators $g_i, g_j \in S$, namely $\theta_{v,\{g_i, g_j\}}$, by $g_{ij} = g_{ji}$. The restrictions:

$$r_i: \sum_{j \neq i} g_{ij}$$

Thus we get

$$\begin{aligned} \sum_i^n r_i &= \sum_i^n \sum_{j \neq i}^{n+1} g_{ij} \\ &= \sum_i^n \sum_{j < i}^n g_{i,j} + g_{j,i} + \sum_i^n g_{i,n+1} \\ &= r_{n+1} \end{aligned}$$

Thus the dimension of the small code at v is at least:

$$\binom{d}{2} - (d - 1) = 3$$

We have $\binom{4}{3} = 4$ stabilizers rooted at v (each for each cube). Yet:

$$s_1 = g_{1,2} + g_{2,3} + g_{1,3}$$

$$s_2 = g_{1,2} + g_{2,4} + g_{1,4}$$

$$s_3 = g_{1,3} + g_{3,4} + g_{1,4}$$

$$s_4 = g_{2,3} + g_{3,4} + g_{2,4}$$

And clearly

$$s_4 = s_1 + s_2 + s_3$$

Definition 18.7 (Reduced subset with boundrey at v).

Definition 18.8 (Base graph G_o). Let $C = \{u_i\}_{i=1}^n$ be a quantum circuit, at depth d , with registers $R_1, \dots, R_k$, all of which are encoded by Toric codes with the same parameters (i.e., dimension, length, etc.). Let's denote by $\mathcal{G}$ the Toric complex that induces the codes, and suppose that the locations of the gates in C are given in registers' coordinate systems. We define the base graph of C , denoted by $G_o = (V_o, E_o)$ as follows:

1. The vertices V_o are tuples, where the first coordinates correspond to a vertex in $\mathcal{G}$, and the second coordinate is associated with a time. Namely:

$$V_o := \bigcup_i V(\mathcal{G}_i) \times [d]$$

2. The edges E_o are derived from the original edges in $\mathcal{G}$, with the addition of edges of the following form: For two vertices at preceding times t and $t + 1$, let's denote them by (v_1, t) and $(v_2, t + 1)$, for which there exist registers R'_1, R'_2 , and faces $f_1, f_2 \in \mathcal{G}$, such that $v_1 \in f_1, v_2 \in f_2$ and C contains a gate that acts non-trivially on the locations $(t, f_1, \mathbf{1}_{R'_1})$ and $(t, f_2, \mathbf{1}_{R'_2})$ at time t , are also connected by an edge. Namely:

$$\begin{aligned} E_o := \{ \{ (v, t), (u, t') \} : & \text{ if} \\ & \text{either } \{v, u\} \in \mathcal{G} \text{ and } t' = t, t + 1 \\ & \text{or there exists } u_i = (g_i, l_i) \in C, f_1, f_2 \in \mathcal{G}, \text{ and } R'_1 R'_2 \\ & \text{such } v \in f_1, u \in f_2, t = l_{i,1}, t' = l_{i,1} + 1 \\ & \text{and } (f_1, \mathbf{1}_{R'_1}), (f_2, \mathbf{1}_{R'_2}) \in l_{i,2} \} \end{aligned}$$